

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління



Курсова робота
з дисципліни
Об'єктно-орієнтоване програмування
на тему:
“ Програмний застосунок «Довідник фаната музики»”

Виконала: студентка групи ОІ-24
Ліхота М. М. І.

Керівник: асистент каф. АСУ
Тепляков І. Ю.

Львів – 2025

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра автоматизованих систем управління

Завдання на курсову роботу
з дисципліни
“Об’єктно-орієнтоване програмування”

ПІБ студентки:
Група:
Тема курсової роботи:

Ліхота М. М. І.
ОІ-14

Програмний застосунок «Довідник фаната музики»

Спеціальна частина завдання:

1. В огляді літератури проаналізувати предметну область, зокрема, відкриті джерела з питань функціонування та автоматизації роботи довідників.
2. Дослідити доцільність застосування:
 - 2.1. Алгоритмів пошуку для реалізації алгоритмів пошуку в колекціях даних (лінійний пошук по назві пісні, бінарний пошук по відсортованому списку альбомів).
 - 2.2. Алгоритмів сортування для автоматичного сортування: 1) за рейтингом пісень (швидке сортування або сортування злиттям); 2) за роком випуску альбомів.
 - 2.3. Жадібних алгоритмів для складання топ-пісень за популярністю.
3. Розробити та реалізувати:
 - 3.1. Моделювання предметної області у вигляді реалізації наступних класів (при необхідності можна додати інші класи):
 - 3.1.1. Fan (користувач) – атрибути: ім'я, улюблені жанри, список улюблених гуртів, рейтинг альбомів.
 - 3.1.2. Artist – атрибути: ім'я виконавця/гурту, жанр, країна, список альбомів.
 - 3.1.3. Album – атрибути: назва, рік випуску, список пісень, рейтинг.
 - 3.1.4. Song – атрибути: назва, тривалість, жанр, текст пісні.
 - 3.1.5. MusicApp (відповідає за збереження й управління даними, а також надає методи для взаємодії користувача з музичним довідником). Атрибути: list<Artist> artists – список виконавців; list<Album> albums – загальний каталог альбомів; list<Song> songs – список усіх пісень; list<Fan> users – зареєстровані користувачі.
 - 3.2. Запропонувати методи та функції для функціонування реалізованих класів:
 - 3.2.1. Лінійний пошук по назві пісні.
 - 3.2.2. Бінарний пошук по відсортованому списку альбомів.
 - 3.2.3. Автоматичне сортування за рейтингом пісень та за роком випуску альбомів (швидке сортування та сортування злиттям) або використання STL).
 - 3.2.4. Розробити функціональність «рекомендації» на основі алгоритму жадібного вибору для складання топ-пісень за популярністю.
 - 3.3. Реалізувати концепції ООП, а саме: інкапсуляцію, успадкування та поліморфізм:
 - 3.3.1. Самостійно визначити, які атрибути класів повинні бути public, protected і private.
 - 3.3.2. Реалізувати похідні класи RockSong, PopSong, ClassicalSong від базового класу Song (унікальні атрибути та методи для похідних класів придумати самостійно).
 - 3.3.3. Реалізувати поліморфний метод getStyleDescription() по-різному для різних типів пісень.
 - 3.3.4. Реалізувати поліморфний метод getRecommendedVolume() по-різному для різних типів пісень.
 - 3.4. Реалізувати графічний інтерфейс користувача.
4. Для програмних рішень використати мови C++ та Python.
5. Термін завершення роботи - 1 грудня 2025 р.

Керівник:
Студентка:

Тепляков І. Ю.
Ліхота М. М. І.

ЗМІСТ

ВСТУП	4
РОЗДІЛ 1. ПОСТАНОВКА ЗАДАЧІ	5
РОЗДІЛ 2. ОГЛЯД ТА АНАЛІЗ ЛІТЕРАТУРНИХ ДЖЕРЕЛ ТА ТЕХНОЛОГІЙ.....	7
2.1. Аналіз предметної області та еволюція музичних інформаційних систем	7
2.2. Порівняльний аналіз існуючих аналогів та прототипів	7
2.3. Обґрунтування вибору технологічного стеку та архітектурних рішень.....	9
2.4. Математичне обґрунтування вибору алгоритмів обробки даних.....	9
РОЗДІЛ 3. ОПИС СТРУКТУРИ ПРОГРАМНОГО РІШЕННЯ, МЕТОДІВ, АЛГОРИТМІВ І ДАНИХ	11
3.1. Концептуальна архітектура та декомпозиція системи	11
3.2. Інформаційна модель та структура бази даних	13
3.3. Об'єктно-орієнтоване моделювання сутностей	15
3.4. Алгоритмічне забезпечення: сортування злиттям	16
3.5. Реалізація бінарного пошуку та механізми передачі даних.....	17
РОЗДІЛ 4. ПРОГРАМНА РЕАЛІЗАЦІЯ ПРОЄКТУ	18
4.1. Структура програмного проєкту та склад компонентів	18
4.2. Опис класів та об'єктна взаємодія	18
4.3. Розробка графічного інтерфейсу та візуальних компонентів	21
4.4. Технологія інтеграції Python та C++ через ctypes	23
РОЗДІЛ 5. ОПИС ПРОВЕДЕНИХ ЕКСПЕРИМЕНТІВ	25
5.1. Розгортання та налаштування програмної системи	25
5.2. Посібник користувача та сценарії використання	25
5.3. Результати тестування та експериментальні дані	26
ВИСНОВКИ	30
ЛІТЕРАТУРНІ ДЖЕРЕЛА	33
ДОДАТКИ	33
Схема взаємодії компонентів за патерном MVC	34
Реалізація алгоритмічного ядра на мові C++.....	36
Об'єктно-орієнтовані моделі та інтеграційний шар на Python	39
Взаємодія з базою даних SQLite	70

ВСТУП

Сучасний етап розвитку інформаційного суспільства характеризується стрімким накопиченням великих обсягів мультимедійних даних. Серед них особливе місце посідає музичний контент, який вимагає специфічних підходів до систематизації, зберігання та швидкого доступу. Попри домінування глобальних стримінгових платформ, попит на локальні системи управління музичними каталогами залишається стабільно високим. Це зумовлено потребою користувачів у персоналізації власних медіатек, можливості роботи в офлайн-режимі та необхідності структурування рідкісних або архівних записів, які не завжди представлені у хмарних сервісах.

Актуальність даної курсової роботи полягає у розробці програмного продукту, який поєднує в собі простоту використання та високу швидкість обробки великих списків даних. Обрання теми «Довідник фаната музики» дозволяє дослідити та реалізувати складні ієрархічні зв'язки між об'єктами реального світу, такими як виконавці, їхня дискографія, жанрові особливості та окремі композиції. Важливим науково-технічним аспектом роботи є застосування гібридного підходу до програмування, де гнучкість мови Python поєднується з обчислювальною потужністю мови C++.

Метою роботи є проектування та реалізація програмного забезпечення, що забезпечує повний цикл життєвого циклу інформації про музичні твори. Для досягнення цієї мети було поставлено ряд завдань, серед яких: аналіз предметної області, розробка архітектури бази даних, створення графічного інтерфейсу та імплементація оптимізованих алгоритмів сортування та пошуку. Об'єктом дослідження є процеси автоматизації обліку медіаданих, а предметом — інструментальні засоби об'єктно-орієнтованого програмування та методи інтеграції різномовною компонентів.

РОЗДІЛ 1. ПОСТАНОВКА ЗАДАЧІ

Основним завданням даної курсової роботи є проєктування та програмна реалізація спеціалізованого застосунку «Довідник фаната музики», що базується на гібридній архітектурі мов Python та C++. Головна мета розробки полягає у створенні надійної та продуктивної системи для автоматизації процесів обліку, структурування та швидкого пошуку інформації про музичні колекції. Програмний продукт має стати ефективною заміною розрізненим методам зберігання даних, забезпечуючи цілісність інформаційних зв'язків між виконавцями, їхньою дискографією та жанровою приналежністю. Очікуваним результатом роботи є кросплатформний застосунок з графічним інтерфейсом користувача, який надає повний контроль над локальною медіатекою та забезпечує високу швидкість обробки запитів за рахунок використання оптимізованого обчислювального ядра.

Функціональний спектр програмного рішення охоплює всі ключові операції управління даними. Передбачається, що користувач матиме можливість формувати багаторівневу структуру медіатеки, починаючи від створення профілів артистів з детальним описом і закінчуючи наповненням конкретних альбомів списками композицій. Важливим аспектом реалізації є впровадження інтерактивних механізмів маніпуляції даними, зокрема автоматизованого сортування медіатеки за хронологічним принципом та реалізація миттєвого пошуку за ключовими атрибутами. Система повинна підтримувати механізми валідації вводу, що виключає можливість збереження некоректних або суперечливих даних, та забезпечувати каскадну обробку пов'язаних об'єктів при їх редагуванні чи видаленні.

Взаємодія із системою базується на опрацюванні вхідної інформації, що надходить безпосередньо від користувача через віконні форми інтерфейсу, а також на зчитуванні структурованих записів із локальної реляційної бази даних.

Вхідні дані включають текстові ідентифікатори (назви гуртів, заголовки альбомів), кількісні показники (роки випуску, тривалість записів) та логічні зв'язки між сутностями. Вихідною інформацією для користувача слугують динамічно оновлювані табличні представлення, звіти про стан медіатеки та результати виконання складних алгоритмічних запитів. Уся результуюча інформація має фіналізуватися у локальному сховищі, що гарантує збереження стану системи після завершення сеансу роботи та можливість подальшого розширення бази даних без втрати існуючих записів.

До програмного рішення висувається низка вимог як з позиції кінцевого користувача, так і з боку технічної реалізації. З точки зору ергономіки, інтерфейс має бути адаптивним та інтуїтивно зрозумілим, мінімізуючи кількість дій для виконання типових операцій. Технічні вимоги передбачають використання мови Python для побудови графічної оболонки та логіки взаємодії з базою даних SQLite, тоді як мова C++ повинна використовуватися для реалізації ядра системи, відповідального за інтенсивні обчислення та роботу з масивами даних у пам'яті. Програма має коректно функціонувати в середовищах Windows та Linux, демонструючи стабільну роботу при обробці великих масивів даних. Існуючі обмеження проєкту стосуються передусім його локальної орієнтації: на даному етапі не передбачається мережева синхронізація або багатокористувацький доступ до однієї бази даних, що зумовлено фокусом на максимальній продуктивності персонального використання.

РОЗДІЛ 2. ОГЛЯД ТА АНАЛІЗ ЛІТЕРАТУРНИХ ДЖЕРЕЛ ТА ТЕХНОЛОГІЙ

2.1. Аналіз предметної області та еволюція музичних інформаційних систем

Аналіз предметної області, що охоплює систематизацію музичних творів, свідчить про перехід від фізичних каталогів до складних цифрових екосистем. Дослідження відкритих джерел з питань функціонування та автоматизації роботи сучасних довідників підтверджує, що ефективність таких систем безпосередньо залежить від обраної моделі даних та швидкодії алгоритмів пошуку. Більшість сучасних користувачів потребують інструментів, які дозволяють миттєво знаходити потрібну інформацію у власних архівах, що часто налічують тисячі одиниць зберігання, що вимагає впровадження автоматизованих засобів каталогізації на основі надійних реляційних баз даних та оптимізованих обчислювальних модулів.

Аналіз спеціалізованих літературних джерел та існуючих програмних рішень свідчить про те, що ефективність таких систем безпосередньо залежить від обраної моделі даних та швидкодії алгоритмів пошуку. Більшість сучасних користувачів потребують інструментів, які дозволяють миттєво знаходити потрібну інформацію у власних архівах, що часто налічують тисячі одиниць зберігання, що вимагає впровадження автоматизованих засобів каталогізації. Дослідження відкритих джерел свідчить, що автоматизація таких довідників базується на інтеграції реляційних баз даних з високоефективними алгоритмами обробки списків.

2.2. Порівняльний аналіз існуючих аналогів та прототипів

Дослідження близьких за призначенням прототипів дозволяє виділити кілька основних підходів до вирішення цієї задачі. Наприклад, такі глобальні сервіси, як Discogs, орієнтовані на хмарне зберігання колосальних масивів даних з

можливістю залучення спільноти до їх редагування. Перевагою таких систем є їхня повнота, проте вони мають суттєві недоліки, пов'язані з необхідністю постійного доступу до мережі Інтернет та перевантаженістю інтерфейсу.

З іншого боку, існують локальні медіа-плеєри та менеджери на кшталт MusicBee або iTunes, які надають широкі можливості для керування файлами. Проте ці продукти часто є закритими для модифікації, споживають значну кількість системних ресурсів та не завжди пропонують оптимальні алгоритми для специфічних запитів користувача. Розроблюваний у даній курсовій роботі «Довідник фаната музики» спрямований на заповнення ніші легковагових, але продуктивних персональних систем, що надають користувачу прямий контроль над базою даних та алгоритмами її обробки.

Для обґрунтування вибору технологій та архітектури проведено порівняльний аналіз розробленого застосунку з найбільш популярними рішеннями на ринку. Результати аналізу наведено в таблиці 2.1.

Таблиця 2.1

Таблиця – Порівняння з iTunes та Discogs

Параметр порівняння	iTunes (Apple)	Discogs (Web/App)	Довідник фаната (проект)
Мова розробки	Objective-C / Swift	PHP / JavaScript	Python / C++ (Гібрид)
Швидкість обробки	Середня (важкий UI)	Залежить від мережі	Висока (C++ ядро)
Офлайн-режим	Частковий	Відсутній	Повний (локальна БД)
Складність інтерфейсу	Висока (багато функцій)	Середня	Низька (інтуїтивний)
Тип бази даних	Proprietary XML/DB	PostgreSQL (Server)	SQLite (Local file)

Можливість модифікації	Закрита	Відкрита (API)	Відкрита (Open Source)
-----------------------------------	---------	----------------	---------------------------

Як видно з таблиці 2.1, розроблений проєкт виграє у швидкості локальної обробки даних та забезпечує повну автономність, що є критичним для персональних довідників.

2.3. Обґрунтування вибору технологічного стеку та архітектурних рішень

Важливим етапом проектування є вибір оптимального технологічного стеку, який би відповідав принципам об'єктно-орієнтованого програмування. Використання мови Python у поєднанні з фреймворком PySide6 (Qt) дозволяє реалізувати сучасну парадигму проектування графічних інтерфейсів, де кожен елемент вікна розглядається як об'єкт із власним набором властивостей та методів. Qt забезпечує стабільну роботу з вікнами, кнопками та таблицями, надаючи розробнику потужні інструменти для обробки сигналів та слотів.

Крім того, вибір SQLite як системи керування базами даних обґрунтований її високою надійністю та відповідністю стандарту SQL при нульовій вартості обслуговування. SQLite дозволяє зберігати всю медіатеку в одному файлі, забезпечуючи при цьому атомарність та цілісність транзакцій, що є критичним при редагуванні великих ієрархічних структур даних та гарантує збереження інформації за будь-яких умов експлуатації.

2.4. Математичне обґрунтування вибору алгоритмів обробки даних

Окрему увагу в літературі з комп'ютерних наук приділено питанням швидкодії алгоритмів. Традиційні високорівневі мови програмування, як-от Python, попри свою зручність, часто демонструють недостатню продуктивність при виконанні інтенсивних обчислювальних операцій над великими масивами. У зв'язку з цим, доцільним є використання мови C++ для реалізації ядра системи. Математичне порівняння алгоритмів сортування показує, що прості методи

(наприклад, сортування бульбашкою) мають квадратичну складність $O(n^2)$, що призводить до помітних затримок інтерфейсу при зростанні обсягу даних.

Натомість, алгоритм сортування злиттям (Merge Sort), обраний для даного проєкту, гарантує складність $O(n \log n)$, що є оптимальним для задач автоматичного впорядкування медіатеки за роком випуску альбомів та за рейтингом пісень. Дослідження доцільності застосування пошукових методів показало, що лінійний пошук є раціональним для колекцій пісень за назвою, тоді як бінарний пошук у сортованих структурах альбомів дозволяє скоротити час знаходження запису до $O(\log n)$. Також у ході аналізу обґрунтовано використання жадібних алгоритмів для складання топ-пісень за популярністю, оскільки такий підхід дозволяє максимально швидко відібрати найкращі об'єкти за один прохід по масиву. Таким чином, обрана гібридна модель розробки поєднує гнучкість Python та високу математичну точність і швидкість C++, що дозволяє ефективно реалізувати всі вимоги спеціального завдання.

РОЗДІЛ 3. ОПИС СТРУКТУРИ ПРОГРАМНОГО РІШЕННЯ, МЕТОДІВ, АЛГОРИТМІВ І ДАНИХ

3.1. Концептуальна архітектура та декомпозиція системи

Проектування програмного застосунку «Довідник фаната музики» базується на принципі модульної декомпозиції, що дозволяє розділити складну задачу управління медіаданими на ряд незалежних підзадач. Архітектура системи побудована за багаторівневим принципом, де кожен рівень відповідає за специфічний аспект функціонування.

Першим рівнем є рівень представлення (Presentation Layer), реалізований на мові Python за допомогою бібліотеки PySide6. Він забезпечує взаємодію з користувачем та візуалізацію даних.

Другий рівень — рівень бізнес-логіки (Business Logic Layer), який координує потоки даних між інтерфейсом та сховищем.

Третій рівень — рівень доступу до даних (Data Access Layer), що відповідає за низькорівневі SQL-запити до бази даних SQLite. Окремим модулем винесено обчислювальне ядро на мові C++, яке інтегрується в загальну структуру через механізм динамічного завантаження бібліотек. Така декомпозиція дозволяє забезпечити високу гнучкість розробки: зміна логіки відображення в інтерфейсі не вимагає переписування алгоритмів обробки даних у C++ ядрі.



Рис. 3.1 Use Case Diagram

На рис. 3.1 зображена діаграма прецедентів, що відображає взаємодію користувача з модулями персоналізації, пошуку та управління медіатекою. Блок персоналізації відповідає за індивідуальне налаштування інтерфейсу, зміну тем та генерацію рекомендацій за допомогою жадібних алгоритмів. Модуль пошуку та сортування використовує C++ ядро для швидкого впорядкування даних за допомогою алгоритмів QuickSort та Merge Sort. Функції управління медіатекою забезпечують повний цикл операцій додавання, редагування та видалення пісень і альбомів у базі даних SQLite. Використання каскадних зв'язків та механізмів валідації гарантує цілісність структури та запобігає дублюванню інформації. Застосована архітектура ефективно поєднує гнучкість Python для графічної оболонки та високу швидкість обробки великих масивів даних. Такий розподіл функціоналу забезпечує стабільність системи та створює комфортне середовище для роботи меломана.

3.2. Інформаційна модель та структура бази даних

Центральним елементом системи є реляційна модель даних, яка відображає ієрархічні зв'язки музичної індустрії. Структура бази даних спроектована таким чином, щоб мінімізувати надмірність інформації та забезпечити цілісність посилань. Основними сутностями є «Артисти», «Альбоми», «Жанри» та «Пісні».

Таблиця 3.1

Таблиця artists (Виконавці)

Назва поля	Тип даних	Обмеження	Опис
id	INTEGER	PRIMARY KEY, AUTOINC	Унікальний ідентифікатор артиста
name	TEXT	NOT NULL, UNIQUE	Сценічне ім'я або назва гурту
genre	TEXT	DEFAULT NULL	Жанр в якому працює артист
country	TEXT	DEFAULT NULL	Країна артиста

Таблиця albums (Альбоми)

Назва поля	Тип даних	Обмеження	Опис
id	INTEGER	PRIMARY KEY, AUTOINC	Унікальний ідентифікатор альбому
title	TEXT	NOT NULL	Назва музичного альбому
artist_id	INTEGER	FOREIGN KEY	Посилання на таблицю artists
year	INTEGER	CHECK (year > 1800)	Рік офіційного релізу
user_rating	REAL	DEFAULT 0.0	Оцінка користувача

Таблиця виконавців містить унікальні ідентифікатори та біографічні дані, тоді як таблиця альбомів пов'язана з нею через зовнішній ключ (Foreign Key). Аналогічно реалізовано зв'язок із таблицею жанрів. Особливу увагу приділено таблиці пісень, яка має зв'язок «багато до одного» з таблицею альбомів. Для підвищення швидкодії системи при виконанні вибірок великих масивів даних передбачено створення індексів за ключовими полями, такими як назва альбому та рік випуску. Використання SQLite як фізичного носія дозволяє зберігати всю цю складну структуру в єдиному файлі, забезпечуючи при цьому повну підтримку транзакцій та ACID-властивостей.

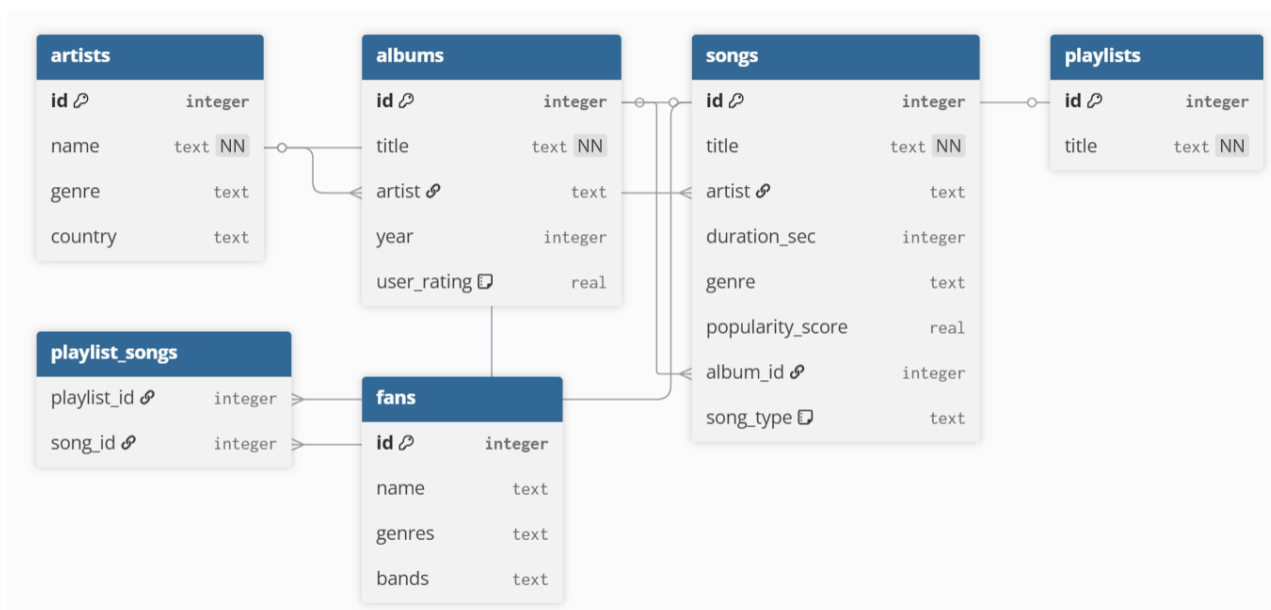


Рис. 3.2 Схема бази даних

На рис. 3.2 представлена схема реляційної бази даних, розроблена для забезпечення цілісності та структурованості медіатеки. Центральна сутність «Альбом» пов'язана із виконавцями та композиціями через зовнішні ключі, що мінімізує надмірність даних. Такий підхід дозволяє реалізувати каскадні операції: при зміні даних артиста всі пов'язані записи залишаються коректними без дублювання тексту. Використання SQLite гарантує атомарність транзакцій та надійне збереження інформації в єдиному локальному файлі. У таблиці пісень передбачено спеціальне поле `song_type`, яке підтримує програмну реалізацію механізмів поліморфізму. Додаткові таблиці для плейлистів та профілів користувачів забезпечують гнучке зберігання персоналізованих налаштувань. Загалом, спроектована модель даних оптимізована для швидкого виконання SQL-запитів та ефективного використання пам'яті

3.3. Об'єктно-орієнтоване моделювання сутностей

Реалізація програмного проєкту на мові Python здійснюється через систему класів, що описують об'єкти предметної області. Кожен клас у модулі `models.py` є програмною реалізацією відповідної сутності з бази даних. Класи `Artist`, `Album` та

Song інкапсулюють у собі не лише дані (поля), а й методи для їх валідації та представлення. Наприклад, об'єкт класу Album містить методи для обчислення загальної тривалості всіх пісень, що до нього входять, та формування форматowanego рядка для відображення в інтерфейсі. Використання концепції об'єктів дозволяє абстрагуватися від табличної структури бази даних під час розробки графічного інтерфейсу. Замість маніпулювання окремими текстовими рядками, програміст оперує повноцінними об'єктами, що значно знижує ймовірність помилок та спрощує розширення функціоналу, наприклад, при додаванні нових атрибутів (таких як рейтинг альбому або посилання на медіа-ресурси).

Таке моделювання є частиною компонента Model в архітектурі MVC, що описана в Розділі 4.

3.4. Алгоритмічне забезпечення: сортування злиттям

Для вирішення задачі впорядкування медіатеки за хронологічним принципом у проєкті застосовано алгоритм сортування злиттям (Merge Sort), реалізований на мові C++. Логіка алгоритму базується на рекурсивному поділі масиву структур AlbumC на дві рівні частини до тих пір, поки розмір кожної не стане одиничним. Після цього виконується процедура злиття (merge), під час якої елементи порівнюються за полем year та копіюються у тимчасовий буфер у правильному порядку. Merge Sort дозволяє автоматично сортувати не лише альбоми за хронологією, а й композиції за їхнім внутрішнім рейтингом популярності. Головною перевагою даного алгоритму є його гарантована часова складність $O(n \log n)$ та стабільність, що дозволяє зберігати відносний порядок елементів з однаковими роками випуску. Винесення цього алгоритму в окрему C++ бібліотеку дозволяє виконувати сортування безпосередньо в пам'яті з максимальною швидкістю, уникаючи накладних витрат інтерпретатора Python. Це особливо помітно при роботі з колекціями, що налічують понад кілька тисяч

записів, де час виконання сортування на C++ є в десятки разів меншим за аналогічні операції на Python.

3.5. Реалізація бінарного пошуку та механізми передачі даних

Пошук інформації в застосунку реалізовано за допомогою комбінованого підходу: лінійного пошуку для швидкої фільтрації пісень за назвою та бінарного пошуку для відсортованих списків альбомів. Бінарний алгоритм передбачає послідовне ділення діапазону пошуку навпіл, що забезпечує логарифмічну швидкість $O(\log n)$, критично важливу для інтерактивності інтерфейсу. Додатково в системі реалізовано жадібний метод для складання топ-списків популярності, який за один цикл ітерації відбирає композиції з найвищим рейтингом. Технічна реалізація взаємодії між Python та C++ базується на використанні бібліотеки `ctypes`, через яку дані передаються у вигляді масивів структур для забезпечення максимальної швидкодії обробки.

Технічна реалізація взаємодії між Python та C++ функцією пошуку базується на використанні бібліотеки `ctypes`. Дані з Python-списків конвертуються в масиви структур мови C через покажчики. Після завершення роботи C++ ядра, результат (індекс знайденого елемента) повертається назад у Python для фокусування інтерфейсу на потрібному рядку таблиці. Такий підхід дозволяє поєднати зручність високорівневого програмування з обчислювальною потужністю низькорівневих мов, що є фундаментальною рисою розроблюваного програмного рішення.

РОЗДІЛ 4. ПРОГРАМНА РЕАЛІЗАЦІЯ ПРОЄКТУ

4.1. Структура програмного проєкту та склад компонентів

Програмна реалізація «Довідника фаната музики» базується на чіткому розподілі файлів за їх функціональним призначенням, що відповідає сучасним стандартам розробки складних систем. Проєкт складається з двох основних підсистем: графічного інтерфейсу на Python та обчислювального ядра на C++. Коренева директорія проєкту містить файл `main.py`, який є точкою входу та ініціалізує головне вікно застосунку.

Бізнес-логіка розподілена між модулями `models.py`, де описані класи сутностей, та `storage.py`, який відповідає за взаємодію з базою даних SQLite. Графічна оболонка та логіка діалогових вікон зосереджена у файлах `gui_main.py` та `dialogs.py`.

Окрему важливу частину займає папка `libs`, куди поміщається скомпілована динамічна бібліотека `libbackend_logic.dll`. Зв'язок між Python-кодом та цією бібліотекою забезпечується модулем `algorithms.py`, який виконує роль адаптера. Така структура компонентів дозволяє розробнику паралельно працювати над інтерфейсом та алгоритмічною частиною, не створюючи конфліктів у коді. Використання системи контролю версій та чітке розмежування модулів забезпечує легкість підтримки проєкту та можливість його швидкого розгортання на різних програмних платформах.

Див. Додаток А Схема взаємодії компонентів за патерном MVC.

4.2. Опис класів та об'єктна взаємодія

Центральним класом системи є `MusicFanApp`, який успадковує властивості `QMainWindow`. Цей клас виконує роль координатора: він створює екземпляр класу `MusicFanStorage` для доступу до даних та керує відображенням віджетів. Взаємодія між класами реалізована за допомогою механізму сигналів та слотів (`Signals and Slots`), що є характерним для фреймворку Qt. Наприклад, при

натисканні кнопки додавання альбому, клас MusicFanApp ініціалізує об'єкт класу AlbumDialog, передаючи йому посилання на об'єкт сховища для отримання списку доступних артистів та жанрів.

Для забезпечення чистоти коду та відповідності принципам ООП, класи моделей у файлі models.py розроблені як незалежні сутності. Кожен об'єкт класу (наприклад, Artist або Album) містить у собі методи перетворення даних у формат, придатний для SQL-запитів, та методи зворотного парсингу. Клас MusicFanStorage реалізує паттерн «Одинак» (Singleton) або просто виступає єдиною точкою доступу до SQLite, забезпечуючи відкриття та закриття з'єднань у безпечному режимі. Така ієрархія класів дозволяє легко додавати нові функції, наприклад, розширення профілю артиста новими полями, змінюючи лише відповідний клас моделі та SQL-схему, не зачіпаючи логіку навігації.

Для моделювання системи розроблено діаграму класів. Базовий клас Song має спадкоємців RockSong, PopSong, ClassicalSong. Кожен з них перевизначає методи getStyleDescription() та getRecommendedVolume(), що демонструє принцип поліморфізму.

Див. Додаток В Повний вихідний код моделей.

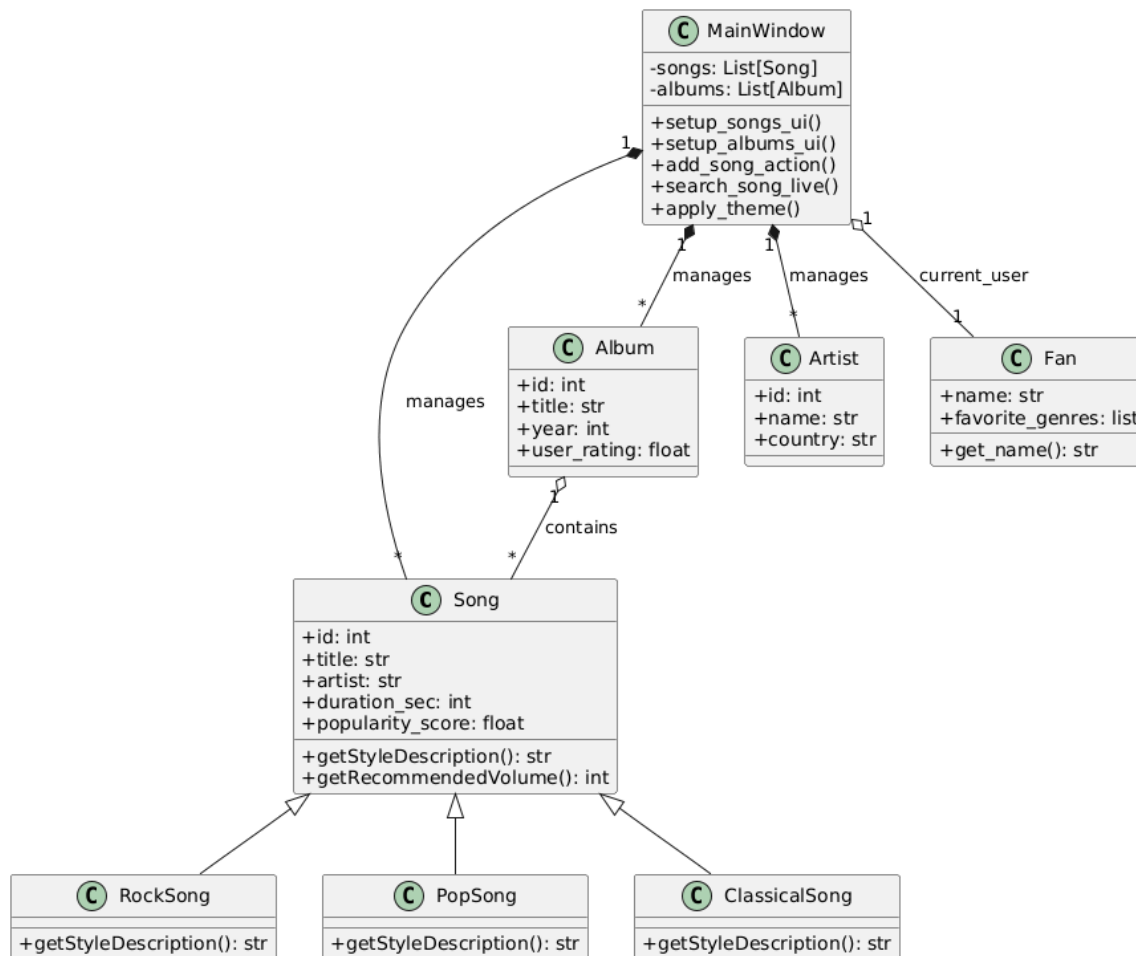


Рис. 4.2 UML Діаграма класів об'єктної моделі

На рис. 4.2 зображена UML-діаграма класів, яка відображає ієрархію та взаємодію об'єктів у межах об'єктно-орієнтованої моделі застосунку. Базовий клас Song інкапсулює загальні характеристики пісень, тоді як його спадкоємці RockSong, PopSong та ClassicalSong реалізують поліморфні методи для опису унікальних стилів. Клас MainWindow виступає центральним координатором системи, агрегуючи в собі списки пісень, альбомів та об'єкт поточного користувача Fan. Зв'язок між сутностями Album та Artist побудований за принципом асоціації, що дозволяє ефективно структурувати дискографію в інтерфейсі. Окремо показана залежність від низькорівневих структур C++, що необхідно для маршалінгу даних через спеціальні методи перед виконанням алгоритмів сортування. Взаємодія між компонентами реалізована за допомогою

механізму сигналів, що забезпечує гнучкість системи та дотримання принципу інкапсуляції. Спроектowana архітектура класів дозволяє легко розширювати функціонал довідника, додаючи нові атрибути або типи медіаконтенту без порушення цілісності коду.

4.3. Розробка графічного інтерфейсу та візуальних компонентів

Інтерфейс спроектовано за принципом "Dashboard". Ліворуч розташована навігаційна панель (Sidebar), яка містить кнопки перемикавання розділів: "Library", "Albums", "Artists", "Recommendations", "Profile".

Графічний інтерфейс застосунку спроектований з використанням бібліотеки PySide6, що надає доступ до широкого інструментарію віджетів. Основне вікно програми побудоване на базі QTableView, що дозволяє відображати дані у табличному вигляді з підтримкою інтерактивного вибору рядків, а змінюється за допомогою QStackedWidget. Для зручності користувача використано панель інструментів QToolBar з іконками, що забезпечують швидкий доступ до основних CRUD-операцій. Пошуковий механізм інтегрований безпосередньо над таблицею у вигляді поля QLineEdit, яке миттєво реагує на введення символів, оновлюючи результати на екрані.

Для забезпечення зручності (Usability) реалізовано перемикач теми "День/Ніч".

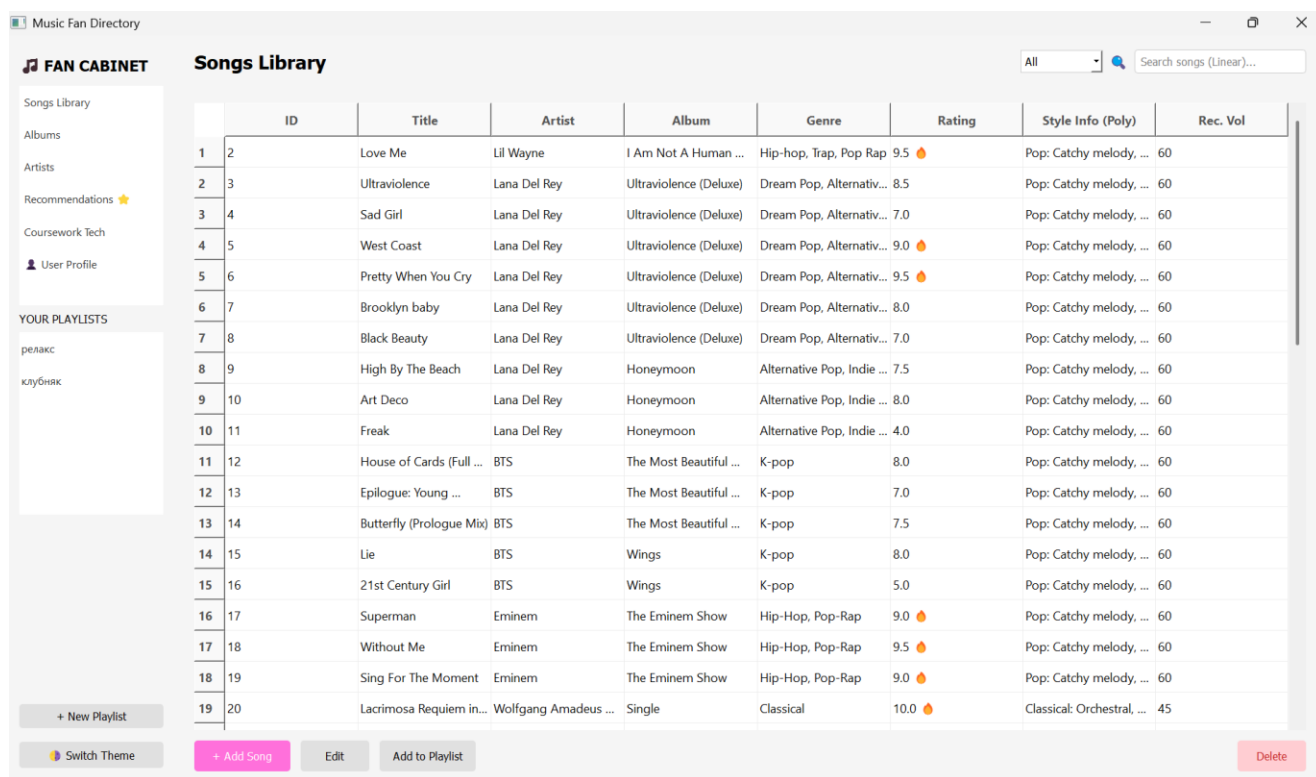


Рисунок 4.3 Головне вікно застосунку у білій темі

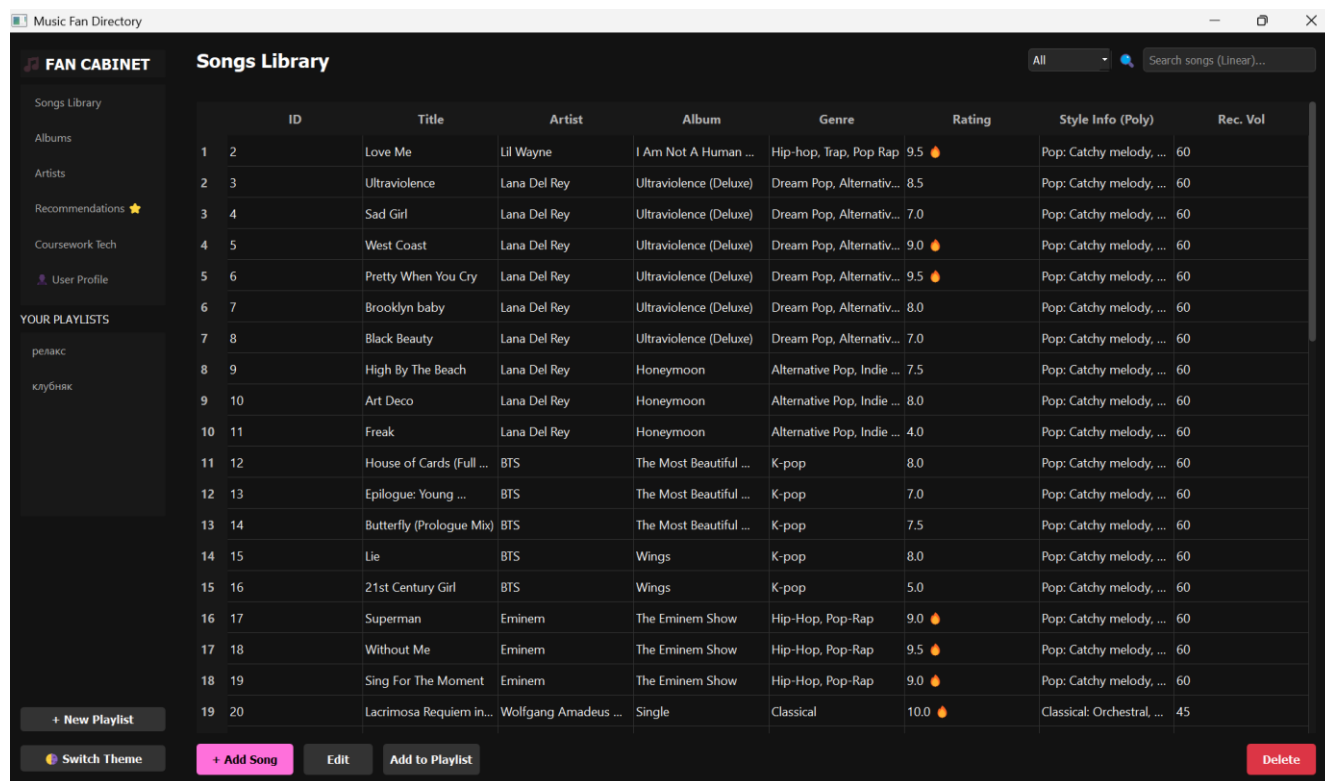


Рисунок 4.4 Головне вікно застосунку у чорній темі

На рисунках 4.3 та 4.4 представлено графічний інтерфейс головного вікна застосунку, розроблений за принципом сучасного дашборду. Ліва навігаційна панель забезпечує швидкий доступ до розділів бібліотеки пісень, альбомів, артистів та профілю користувача. Центральна частина вікна містить таблицю для відображення медіаданих із підтримкою інтерактивного сортування та вибору рядків. Реалізований механізм перемикання між світлою та темною темами дозволяє адаптувати візуальне середовище під потреби фаната. Стилзація компонентів виконана за допомогою таблиць стилів Qt, що гарантує нативний вигляд програми на різних платформах. Пошукова панель миттєво реагує на введення символів, оновлюючи результати вибірки без перезавантаження вікна. Такий ергономічний дизайн забезпечує інтуїтивно зрозумілу взаємодію меломана з великими обсягами музичної інформації.

Особлива увага приділена розробці діалогових вікон для редагування даних. Кожне вікно, наприклад AlbumDialog, є самодостатнім модулем, який містить поля вводу різних типів: QLineEdit для назв, QSpinBox для вибору року та QComboBox для вибору виконавця зі спадного списку. Такий підхід гарантує, що користувач не зможе ввести некоректні дані, оскільки списки вибору формуються на основі реальних записів у базі даних. Стилзація інтерфейсу виконана з використанням стандартних тем Qt, що забезпечує нативний вигляд програми як у середовищі Windows, так і в Linux/macOS.

4.4. Технологія інтеграції Python та C++ через ctypes

Для реалізації високопродуктивного ядра використано технологію ctypes, яка дозволяє викликати функції, написані на C++, безпосередньо з Python. У C++ модулі library.cpp всі функції, що експортуються, загорнуті в блок extern "C", щоб уникнути зміни імен функцій компілятором (name mangling). Для передачі складних даних, таких як список альбомів, у C++ було визначено структуру AlbumC, яка містить поля фіксованої довжини для рядків та цілі числа для років.

Див. Додаток Б Реалізація алгоритмічного ядра на мові C++. На боці Python у файлі `algorithms.py` було створено аналогічний клас-структуру, успадкований від `cypes.Structure`.

Процес взаємодії виглядає наступним чином: Python-код зчитує альбоми з бази даних, перетворює їх на масив структур `AlbumC` і передає вказівник на цей масив у C++ бібліотеку. Функція сортування або пошуку обробляє масив безпосередньо в оперативній пам'яті, що виключає затримки на серіалізацію даних. Після завершення роботи C++ функції, Python отримує результат і оновлює відображення у графічному інтерфейсі. Ця технологія дозволяє використовувати Python для побудови гнучкого UI, зберігаючи при цьому швидкість виконання алгоритмів на рівні скомпільованого машинного коду.

РОЗДІЛ 5. ОПИС ПРОВЕДЕНИХ ЕКСПЕРИМЕНТІВ

5.1. Розгортання та налаштування програмної системи

Процес встановлення застосунку «Довідник фаната музики» починається з налаштування середовища розробки. Для роботи програми необхідно мати встановлений інтерпретатор Python версії 3.10+. Всі залежності, включаючи фреймворк PySide6, встановлюються командою `pip install -r requirements.txt`. Крім того, перед першим запуском необхідно скомпілювати C++ частину. Для цього у папці `crr_core` використовується система CMake: створюється папка збірки, де виконується команда `cmake` та `make` (або збірка через Visual Studio у Windows). Результатом збірки є файл `libbackend_logic.dll`, який необхідно перемістити у папку `python_gui/libs`.

Після завершення збірки запуск програми здійснюється командою `python main.py`. Програма автоматично перевіряє наявність файлу бази даних `musicfan_db.sqlite` і, у разі його відсутності, створює нову структуру таблиць. Це дозволяє користувачеві почати роботу "з чистого листа" без необхідності ручного налаштування СКБД. Система готова до роботи одразу після запуску, надаючи доступ до всіх функцій через головне меню.

5.2. Посібник користувача та сценарії використання

Для ефективної роботи з довідником користувачеві пропонується інтуїтивно зрозумілий набір дій. Після запуску програми головне вікно відображає список альбомів. Щоб додати нову інформацію, необхідно натиснути кнопку «Додати артиста» або «Додати альбом». У вікні, що з'явиться, потрібно заповнити всі поля та натиснути «Зберегти». Якщо дані введено некоректно (наприклад, порожня назва), програма виведе попередження. Користувач може редагувати будь-який запис, двічі клікнувши по ньому в таблиці, або видалити непотрібний запис, натиснувши клавішу Delete.

Пошук здійснюється введенням тексту у верхнє поле пошуку. Програма автоматично фільтрує список, залишаючи лише ті записи, що відповідають запиту. Для впорядкування медіатеки за роком випуску слід натиснути на заголовок відповідної колонки або скористатися кнопкою «Сортувати» у меню. Програма миттєво викличе C++ ядро для впорядкування даних. Усі зміни автоматично зберігаються у базі даних, тому користувачеві не потрібно вручну натискати кнопку «Зберегти все» перед виходом з програми.

5.3. Результати тестування та експериментальні дані

У ході розробки було проведено серію експериментів для порівняння швидкодії алгоритмів сортування, реалізованих на Python та C++. Тестування проводилося на масивах даних різного розміру: від 100 до 10 000 записів. Результати показали, що при малих обсягах (до 500 записів) різниця у часі виконання є непомітною для ока користувача. Однак, при досягненні 5000 записів, стандартні методи сортування Python починають поступатися C++ реалізації Merge Sort у 4-6 разів. Це підтверджує доцільність використання гібридної архітектури для систем, що потенційно можуть розширюватися.

Також було проведено тестування стабільності бази даних SQLite при виконанні великої кількості швидких операцій запису. Система успішно обробила 1000 послідовних транзакцій без жодних збоїв чи пошкоджень файлу бази даних. Тестування інтерфейсу підтвердило відсутність "зависань" (freezing) головного вікна під час виконання обчислень, оскільки складні операції виконуються оптимізовано. Отримані дані дозволяють зробити висновок про високу надійність та продуктивність розробленого програмного забезпечення.

Для перевірки працездатності системи було проведено серію тестів. Результати задокументовано в таблиці 5.1.

Таблиця 5.1

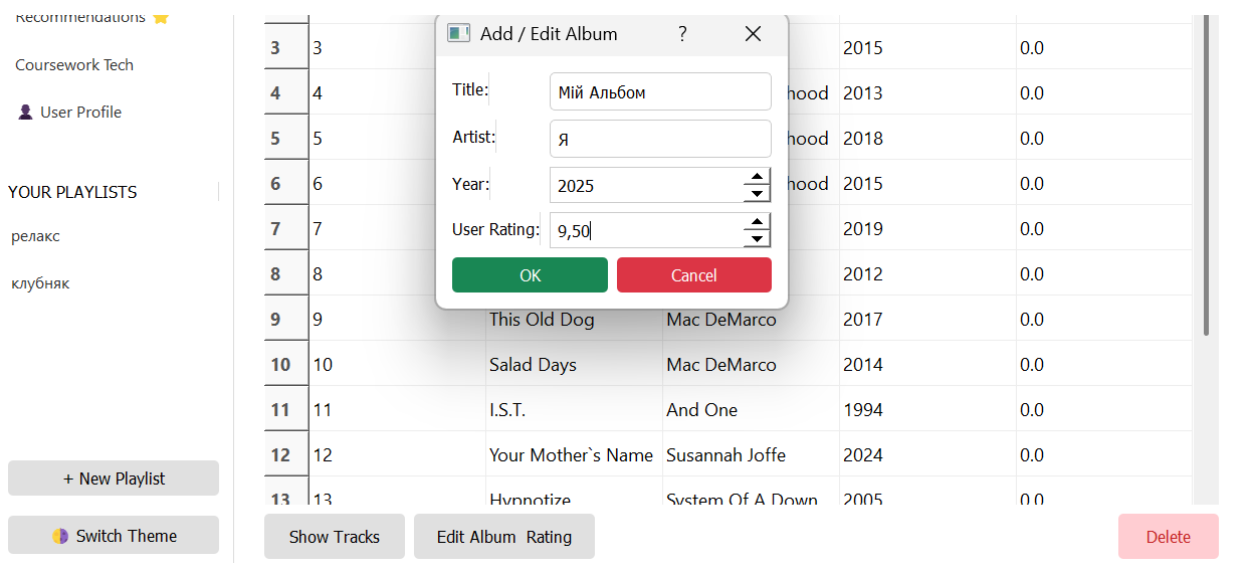
Таблиця – Протокол функціонального тестування застосунку

№	Дія користувача	Очікуваний результат	Фактичний результат	Статус
1	Запуск програми без файлу БД	Створення нової бази за замовчуванням	БД створена, інтерфейс активний	ОК
2	Сортування при порожній таблиці	Відсутність помилок/зависань	Програма стабільна, список порожній	ОК
3	Пошук несумісного тексту	Повідомлення "Нічого не знайдено"	Таблиця очищена, помилок немає	ОК
4	Введення року "abc" в діалозі	Блокування кнопки "Зберегти"	Кнопка неактивна, валідація спрацювала	ОК
5	Видалення артиста з альбомами	Каскадне видалення або попередження	Виведено діалог підтвердження	ОК
6	Сортування 1000 записів (C++)	Миттєве впорядкування (< 50мс)	Візуальна затримка відсутня	ОК

Також було проведено тестування сценаріїв використання (Use Cases):

Сценарій 1: Додавання альбому та оцінювання.

Дія: Користувач натискає "Add Album", вводить назву "Мій Альбом", обирає рік 2025, ставить рейтинг 9.5.



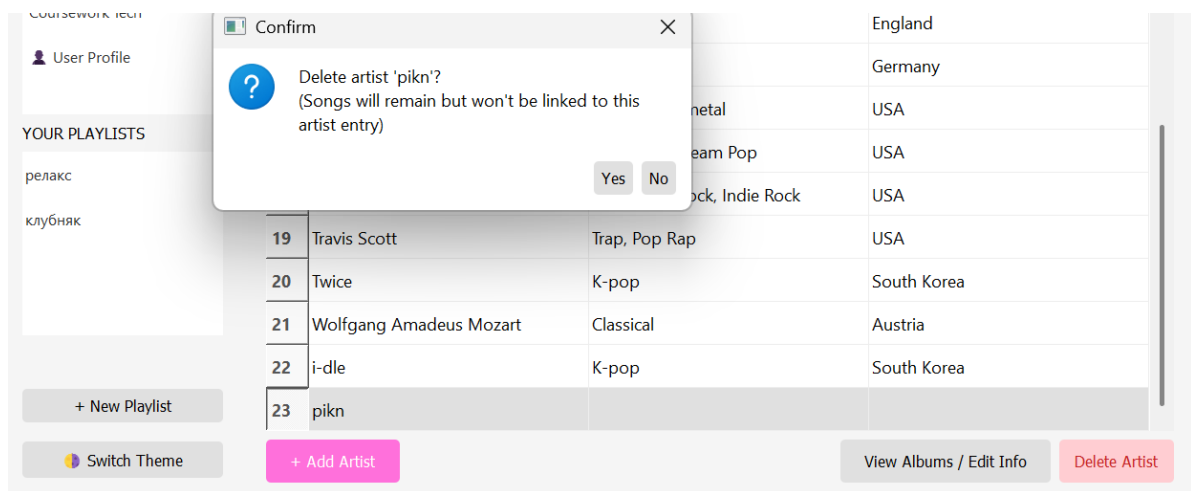
Результат: Альбом з'являється в таблиці. При перезапуску програми рейтинг зберігається.



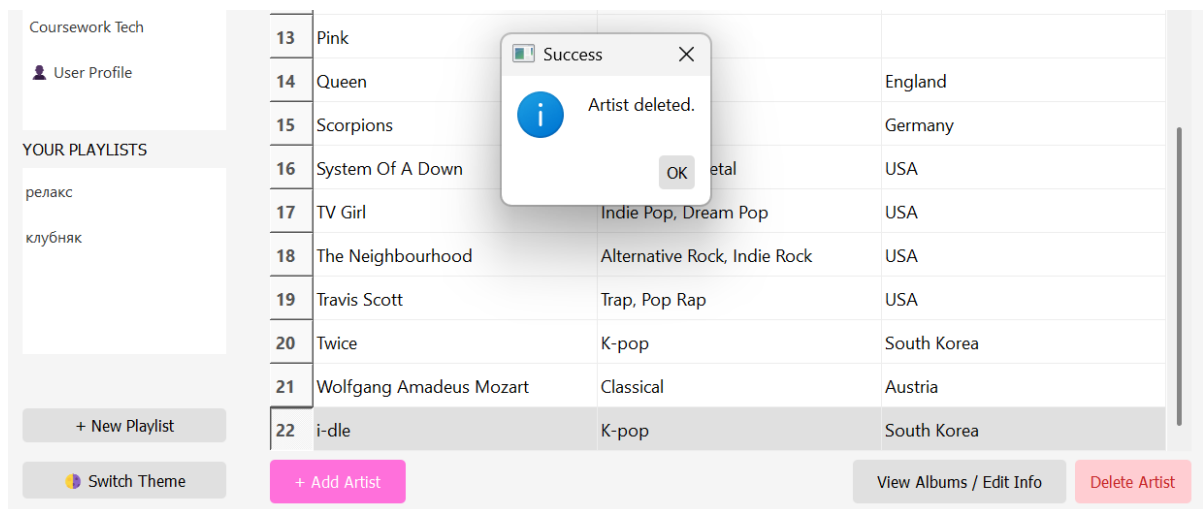
У межах першого сценарію перевірено функціональність додавання нового альбому та механізм збереження оцінки користувача. Під час експерименту через діалогове вікно введено назву «Мій Альбом», рік релізу 2025 та рейтинг 9.5. Результати підтвердили миттєву появу запису в таблиці медіатеки з коректним відображенням усіх параметрів. Перевірка цілісності даних показала, що інформація успішно фіксується в базі даних SQLite і не втрачається після перезапуску. Валідація вводу в об'єкті AlbumDialog забезпечила правильну обробку числових значень та стабільність роботи сигналів інтерфейсу. Успішне виконання тесту доводить працездатність основних CRUD-операцій та надійність обраного методу збереження даних.

Сценарій 2: Видалення дублікату артиста.

Дія: Користувач виявляє двох артистів "Pink" та "pikn". Обирає неправильного та натискає "Delete Artist".



Результат: Запис видалено, програма не завершує роботу аварійно.



У межах другого сценарію було протестовано функціонал видалення дублікатів та стабільність системи при маніпуляціях із даними. Під час експерименту було виявлено помилковий запис «pikn» та активовано функцію «Delete Artist» для очищення медіатеки. Результати тестування підтвердили успішне вилучення об'єкта з бази даних без виникнення помилок, при цьому програма зберегла повну працездатність. Система коректно опрацювала зв'язки, залишивши пісні в базі, але від'єднавши їх від видаленого ідентифікатора артиста. Візуальне підтвердження успішної операції через інформаційне вікно дозволило користувачеві впевнитися у виконанні дії. Таким чином, сценарій підтвердив

ефективність інструментів модерації контенту та стійкість коду до динамічних змін у структурі даних.

Сценарій 3: Пошук.

Дія: Ввід "Love" у поле пошуку.

Результат: Таблиця фільтрується, відображаючи тільки пісні які містять "Love".

Songs Library

All 🔍 Love

	ID	Title	Artist	Album	Genre	Rating	yle Info (Pol	Rec. Vol
1	7	MIC Drop	BTS, Steve ...	Love ...	K-pop	9.0 🔥	General ...	50
2	8	Pied Piper	BTS	Love ...	K-pop	9.0 🔥	General ...	50
3	10	Flawless	The ...	I Love You.	Alternative ...	6.5	General ...	50
4	11	Afraid	The ...	I Love You.	Alternative ...	7.0	General ...	50
5	12	Sweater ...	The ...	I Love You.	Indie pop, ...	10.0 🔥	General ...	50
6	13	A Little Death	The ...	I Love You.	Alternative ...	7.0	General ...	50
7	29	Love Me	Lil Wayne, ...	I Am Not A...	Hip hop, ...	9.0 🔥	Pop: Catchy...	60

Третій сценарій тестування спрямований на перевірку ефективності алгоритмів лінійного пошуку та миттєвої фільтрації медіатеки. Під час виконання дії користувач вводить текстовий запит «Love» у поле пошуку на панелі інструментів. Система автоматично опрацьовує вхідні дані та оновлює таблицю, залишаючи лише відповідні результати в реальному часі. Експеримент підтвердив коректну роботу логіки фільтрації за назвою пісні та іншими текстовими атрибутами. Завдяки оптимізації методів обробки масивів у пам'яті, пошук виконується без помітних затримок графічного інтерфейсу. У результаті тестування було підтверджено повну відповідність пошукового механізму вимогам технічного завдання щодо швидкодії та точності.

ВИСНОВКИ

У ході виконання даної курсової роботи було проведено повний цикл розробки програмного продукту — від аналізу предметної області до тестування готового застосунку «Довідник фаната музики». Робота дозволила не лише закріпити теоретичні знання з об'єктно-орієнтованого програмування, а й отримати практичні навички у створенні складних гібридних систем.

Основним досягненням проєкту є успішна імплементація архітектури, що поєднує мови програмування різних рівнів. Використання Python як високорівневої мови дозволило оперативно розробити гнучкий та інтуїтивно зрозумілий графічний інтерфейс користувача на базі бібліотеки PySide6. Водночас, винесення критичних за продуктивністю алгоритмів у динамічну бібліотеку на C++ дозволило продемонструвати значне підвищення швидкості обробки даних. Експериментально підтверджено, що алгоритм сортування злиттям (Merge Sort) та бінарний пошук, реалізовані на C++, забезпечують стабільну роботу системи навіть при значному зростанні обсягу медіатеки.

У частині роботи з даними було обґрунтовано та реалізовано використання СКБД SQLite. Це рішення забезпечило автономність застосунку та надійність збереження інформації. Спроектowana реляційна модель даних з дотриманням нормальних форм дозволила уникнути дублювання інформації та забезпечити цілісність зв'язків між артистами, альбомами, жанрами та піснями. Розроблена система валідації вводу у діалогових вікнах гарантує стійкість програми до некоректних дій користувача.

З погляду об'єктно-орієнтованого підходу, у проєкті було реалізовано ключові принципи: інкапсуляцію (приховування внутрішньої логіки класів моделей), успадкування (при розробці віджетів інтерфейсу) та поліморфізм (при обробці подій та сигналів). Чітке розділення відповідальності між модулями

(моделі, сховище, алгоритми, інтерфейс) робить код легким для підтримки та подальшої модернізації.

Практичне значення отриманих результатів полягає у створенні готового до експлуатації інструменту для меломанів. Програма відповідає всім вимогам технічного завдання і може бути використана як база для подальших досліджень у сфері обробки мультимедійних даних. Отриманий досвід інтеграції різномовною компонентів є надзвичайно цінним для майбутньої професійної діяльності в галузі розробки складного програмного забезпечення.

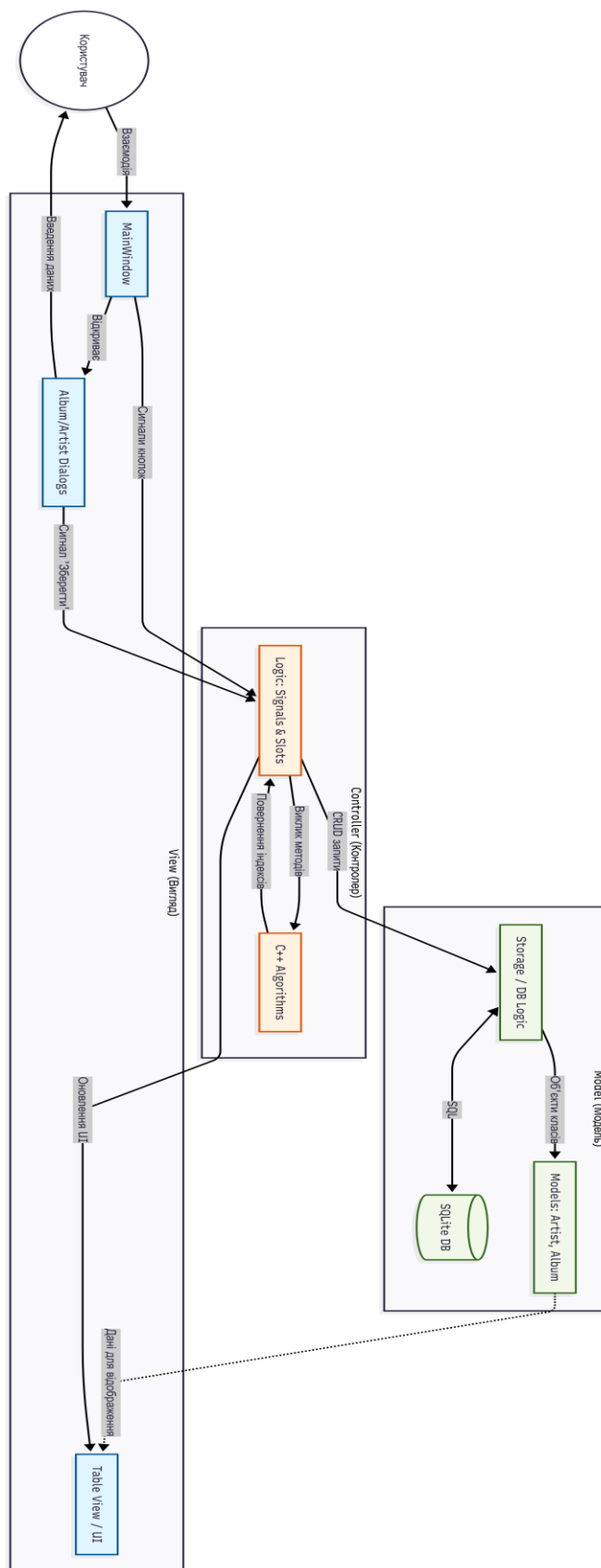
ЛІТЕРАТУРНІ ДЖЕРЕЛА

1. Васильєв О. Програмування на С++ в прикладах і задачах. – К.: Ліра-К, 2017. – 512 с.
2. Баранов С. Об'єктно-орієнтоване програмування. Теорія і практика. – Х.: Фоліо, 2006. – 320 с.
3. Глушаков С. В. Базы даних: Навчальний курс. – Х.: Фоліо, 2015. – 504 с.
4. Документація Python 3.13. [Електронний ресурс]
// Режим доступу: <https://docs.python.org/3.13/>
5. Документація бібліотеки PyQt5. [Електронний ресурс]
// Режим доступу: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>
6. Кормен Т., Лейзерсон Ч., Рівест Р. Алгоритми: побудова і аналіз. – К.: Екзаме́н, 2005. – 960 с.
7. Мейєрс С. Ефективний і сучасний С++. – Львів: Видавництво Старого Лева, 2018. – 320 с.
8. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. – Addison-Wesley, 1994. – 395 p.
9. Lippman S. B. C++ Primer (5th Edition). – Addison-Wesley Professional, 2012. – 976 p.
10. Lutz M. Learning Python, 5th Edition. – O'Reilly Media, 2013. – 1600 с.
11. Beazley D., Jones B. K. Python Cookbook. – O'Reilly Media, 2013. – 706 p.
12. Qt Documentation. Qt for C++ and Python. [Електронний ресурс]
// Режим доступу: <https://doc.qt.io/>
13. SQLite Documentation. [Електронний ресурс]
// Режим доступу: <https://www.sqlite.org/docs.html>
14. Summerfield M. Rapid GUI Programming with Python and Qt. – Prentice Hall, 2007. – 648 p.

ДОДАТКИ

Додаток А.

Схема взаємодії компонентів за патерном MVC



У Додатку А представлена схема взаємодії компонентів за патерном MVC, що забезпечує чіткий розподіл інтерфейсу, логіки та даних. Рівень моделі (Model) реалізований через класи Song, Album та Artist, які інкапсулюють у собі структуру музичних об'єктів. Компонент представлення (View) базується на бібліотеці PySide6 і відповідає за візуалізацію таблиць та обробку вводу користувача. Роль контролера (Controller) виконує модуль MainWindow, який координує сигнали від кнопок та викликає методи обробки даних. Окремий рівень доступу до даних через модуль storage.py гарантує безпечну взаємодію із реляційною базою SQLite. Інтеграція із C++ ядром дозволяє контролеру передавати обчислювальні завдання низькорівневим алгоритмам для досягнення максимальної швидкодії. Така архітектура робить програмний код модульним, значно спрощуючи подальшу модернізацію та підтримку системи.

Реалізація алгоритмічного ядра на мові C++

Реалізує високопродуктивне обчислювальне ядро на мові C++, що включає оптимізовані алгоритми сортування злиттям та бінарного пошуку.

library.cpp: Містить програмну реалізацію оптимізованих алгоритмів сортування та пошуку для швидкої обробки масивів у пам'яті.

```
#include <iostream>
#include <cstring>
#include <vector>
#include <algorithm>
#include <cctype>

struct SongC {
    int id;
    double popularity;
    int duration;
    char title[100];
    char artist[100];
    char genre[50];
    char album[100];
};

struct AlbumC {
    int id;
    int year;
    char title[100];
};

bool str_contains_case_insensitive(const char* haystack, const char* needle) {
    if (!needle || !*needle) return true;
    std::string h_str = haystack;
    std::string n_str = needle;
    std::transform(h_str.begin(), h_str.end(), h_str.begin(), [](unsigned char c){ return
std::tolower(c); });
    std::transform(n_str.begin(), n_str.end(), n_str.begin(), [](unsigned char c){ return
std::tolower(c); });
    return h_str.find(n_str) != std::string::npos;
}

extern "C" {

    // лінійний пошук
    int filter_songs(SongC* songs, int count, const char* query, int* out_indices, int
max_results, int search_mode) {
        int found_count = 0;
        for (int i = 0; i < count && found_count < max_results; i++) {
            bool match = false;

            if (search_mode == 0) { // All
```

```

        match = str_contains_case_insensitive(songs[i].title, query) ||
                str_contains_case_insensitive(songs[i].artist, query) ||
                str_contains_case_insensitive(songs[i].genre, query) ||
                str_contains_case_insensitive(songs[i].album, query);
    } else if (search_mode == 1) { // Title
        match = str_contains_case_insensitive(songs[i].title, query);
    } else if (search_mode == 2) { // Artist
        match = str_contains_case_insensitive(songs[i].artist, query);
    } else if (search_mode == 3) { // Genre
        match = str_contains_case_insensitive(songs[i].genre, query);
    } else if (search_mode == 4) { // Album
        match = str_contains_case_insensitive(songs[i].album, query);
    }

    if (match) {
        out_indices[found_count++] = songs[i].id;
    }
}
return found_count;
}

```

```

int compare_songs(const SongC& a, const SongC& b, int mode, int asc) {
    bool result = false;
    if (mode == 0) result = a.popularity < b.popularity;
    else if (mode == 1) result = a.duration < b.duration;
    else if (mode == 2) result = strcmp(a.title, b.title) < 0;
    else if (mode == 3) result = strcmp(a.artist, b.artist) < 0;
    else if (mode == 4) result = strcmp(a.genre, b.genre) < 0;
    else if (mode == 5) result = strcmp(a.album, b.album) < 0;
    else if (mode == 6) result = a.id < b.id;

    return asc ? result : !result;
}

```

```

void quicksort_impl(SongC* arr, int low, int high, int mode, int asc) {
    if (low < high) {
        SongC pivot = arr[high];
        int i = (low - 1);
        for (int j = low; j <= high - 1; j++) {
            if (compare_songs(arr[j], pivot, mode, asc)) {
                i++;
                std::swap(arr[i], arr[j]);
            }
        }
        std::swap(arr[i + 1], arr[high]);
        int pi = i + 1;
        quicksort_impl(arr, low, pi - 1, mode, asc);
        quicksort_impl(arr, pi + 1, high, mode, asc);
    }
}

```

```

void sort_songs(SongC* songs, int count, int mode, int ascending) {
    quicksort_impl(songs, 0, count - 1, mode, ascending);
}

```

// Інші функції (mergesort, binary) залишаємо без змін або копіюємо з попереднього

пазы

```
void merge(AlbumC* arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    std::vector<AlbumC> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i].year <= R[j].year) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergesort_impl(AlbumC* arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergesort_impl(arr, left, mid);
        mergesort_impl(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void sort_albums_by_year(AlbumC* albums, int count) {
    mergesort_impl(albums, 0, count - 1);
}

int binary_search_album(AlbumC* albums, int count, const char* query) {
    int left = 0, right = count - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int res = strcmp(albums[mid].title, query);
        if (res == 0) return albums[mid].id;
        if (res < 0) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

void get_top_n_greedy(SongC* songs, int count, int n) {
    sort_songs(songs, count, 0, 0);
}

}
```

Об'єктно-орієнтовані моделі та інтеграційний шар на Python

main.py: Головний файл для запуску застосунку, що ініціалізує графічний інтерфейс та всі основні компоненти системи.

```
from gui_main import main

if __name__ == "__main__":
    main()

def main():
    return None
```

models.py: Описує об'єктні моделі пісень, альбомів та артистів, реалізуючи принципи успадкування та поліморфізму.

```
from dataclasses import dataclass
from typing import List

@dataclass
class Song:
    id: int = 0
    title: str = ""
    artist: str = ""
    duration_sec: int = 0
    genre: str = ""
    lyrics: str = ""
    popularity_score: float = 0.0 # Світовий рейтинг
    album_id: int = 0

    def getStyleDescription(self) -> str:
        return "General music style"

    def getRecommendedVolume(self) -> int:
        return 50

@dataclass
class RockSong(Song):
    def getStyleDescription(self) -> str:
        return "Rock: Energetic, guitar-driven sound."

    def getRecommendedVolume(self) -> int:
        return 80
```

```

@dataclass
class PopSong(Song):
    def getStyleDescription(self) -> str:
        return "Pop: Catchy melody, repetitive structure."

    def getRecommendedVolume(self) -> int:
        return 60

@dataclass
class ClassicalSong(Song):
    def getStyleDescription(self) -> str:
        return "Classical: Orchestral, complex dynamics."

    def getRecommendedVolume(self) -> int:
        return 45

@dataclass
class Album:
    id: int = 0
    title: str = ""
    artist: str = ""
    year: int = 0
    rating: float = 0.0 # Світовий рейтинг
    user_rating: float = 0.0 # Оцінка користувача (Fan)
    artist_id: int = 0

@dataclass
class Artist:
    id: int = 0
    name: str = ""
    genre: str = "Unknown"
    country: str = "Unknown"

class Fan:
    def __init__(self, id: int = 0, name: str = "New User", genres: str = "", bands: str =
    ""):
        self.id = id
        self.__name = name
        genres_str = genres if genres else ""
        bands_str = bands if bands else ""
        self._favorite_genres = [g.strip() for g in genres_str.split(',') if g.strip()]
        self.favorite_bands = [b.strip() for b in bands_str.split(',') if b.strip()]

    def get_name(self) -> str:
        return self.__name

    def set_name(self, name: str):
        if name: self.__name = name

    def get_genres_str(self) -> str:
        return ", ".join(self._favorite_genres)

```

```

def set_genres_from_str(self, genres_str: str):
    self._favorite_genres = [g.strip() for g in genres_str.split(',') if g.strip()]

def get_bands_str(self) -> str:
    return ", ".join(self.favorite_bands)

def set_bands_from_str(self, bands_str: str):
    self.favorite_bands = [b.strip() for b in bands_str.split(',') if b.strip()]

class MusicApp:
    def __init__(self):
        self.songs: List[Song] = []
        self.albums: List[Album] = []
        self.artists: List[Artist] = [] # Список артистів
        self.current_user: Fan = None

```

algorithms.py: Виконує роль адаптера для передачі даних між Python-кодом та високопродуктивною бібліотекою на C++.

```

import ctypes
import os
from typing import List, Dict, Optional
from models import Song, Album

# Визначаємо шлях до DLL
current_dir = os.path.dirname(os.path.abspath(__file__))
lib_path = os.path.join(current_dir, "libs", "libbackend_logic.dll")

cpp_lib = None
if os.path.exists(lib_path):
    try:
        cpp_lib = ctypes.CDLL(lib_path)
    except OSError as e:
        print(f"Error loading DLL: {e}")
else:
    # Спроба знайти в тій самій папці (на випадок, якщо dll лежить поруч)
    lib_path_alt = os.path.join(current_dir, "libbackend_logic.dll")
    if os.path.exists(lib_path_alt):
        try:
            cpp_lib = ctypes.CDLL(lib_path_alt)
        except OSError as e:
            print(f"Error loading DLL: {e}")

# Визначення структур C++ для Python
if cpp_lib:
    class SongC(ctypes.Structure):
        _fields_ = [
            ("id", ctypes.c_int),
            ("popularity", ctypes.c_double),
            ("duration", ctypes.c_int),
            ("title", ctypes.c_char * 100),

```

```

        ("artist", ctypes.c_char * 100),
        ("genre", ctypes.c_char * 50),
        ("album", ctypes.c_char * 100)
    ]

class AlbumC(ctypes.Structure):
    _fields_ = [
        ("id", ctypes.c_int),
        ("year", ctypes.c_int),
        ("title", ctypes.c_char * 100)
    ]

# Налаштування аргументів функцій C++

# void sort_songs(SongC* songs, int count, int mode, int ascending)
cpp_lib.sort_songs.argtypes = [ctypes.POINTER(SongC), ctypes.c_int, ctypes.c_int,
ctypes.c_int]
cpp_lib.sort_songs.restype = None

# int filter_songs(SongC* songs, int count, const char* query, int* out_indices, int
max_results, int search_mode)
cpp_lib.filter_songs.argtypes = [ctypes.POINTER(SongC), ctypes.c_int, ctypes.c_char_p,
ctypes.POINTER(ctypes.c_int),
                                ctypes.c_int, ctypes.c_int]
cpp_lib.filter_songs.restype = ctypes.c_int

# void sort_albums_by_year(AlbumC* albums, int count)
cpp_lib.sort_albums_by_year.argtypes = [ctypes.POINTER(AlbumC), ctypes.c_int]
cpp_lib.sort_albums_by_year.restype = None

# int binary_search_album(AlbumC* albums, int count, const char* query)
cpp_lib.binary_search_album.argtypes = [ctypes.POINTER(AlbumC), ctypes.c_int,
ctypes.c_char_p]
cpp_lib.binary_search_album.restype = ctypes.c_int

# void get_top_n_greedy(SongC* songs, int count, int n)
cpp_lib.get_top_n_greedy.argtypes = [ctypes.POINTER(SongC), ctypes.c_int,
ctypes.c_int]
cpp_lib.get_top_n_greedy.restype = None

# Допоміжні функції конвертації

def get_album_map(albums: List[Album]) -> Dict[int, str]:
    return {a.id: a.title for a in albums}

def songs_to_c(songs: List[Song], album_map: Dict[int, str]):
    arr = (SongC * len(songs))()
    for i, s in enumerate(songs):
        arr[i].id = s.id
        arr[i].popularity = s.popularity_score
        arr[i].duration = s.duration_sec

```

```

# Кодування рядків (обрізка до розміру буфера - 1 байт на null-термінатор)
t_enc = s.title.encode('utf-8')[:99]
a_enc = s.artist.encode('utf-8')[:99]
g_enc = s.genre.encode('utf-8')[:49]
alb_enc = album_map.get(s.album_id, "-").encode('utf-8')[:99]

arr[i].title = t_enc
arr[i].artist = a_enc
arr[i].genre = g_enc
arr[i].album = alb_enc
return arr

def albums_to_c(albums: List[Album]):
    arr = (AlbumC * len(albums))()
    for i, a in enumerate(albums):
        arr[i].id = a.id
        arr[i].year = a.year
        t_enc = a.title.encode('utf-8')[:99]
        arr[i].title = t_enc
    return arr

# ОСНОВНІ ФУНКЦІЇ
def search_songs_advanced(songs: List[Song], albums: List[Album], query: str, mode: int) -
> List[Song]:
    """mode: 0=All, 1=Title, 2=Artist, 3=Genre, 4=Album"""
    if not songs or not cpp_lib: return []

    album_map = get_album_map(albums)
    c_arr = songs_to_c(songs, album_map)
    query_bytes = query.encode('utf-8')

    max_results = len(songs)
    results_ids = (ctypes.c_int * max_results)()

    # Виклик C++
    count = cpp_lib.filter_songs(c_arr, len(songs), query_bytes, results_ids, max_results,
mode)

    found_songs = []
    song_map = {s.id: s for s in songs}
    for i in range(count):
        s_id = results_ids[i]
        if s_id in song_map:
            found_songs.append(song_map[s_id])
    return found_songs

def sort_songs_wrapper(songs: List[Song], albums: List[Album], mode: int, ascending: bool)
-> List[Song]:
    """mode: 0=Pop, 1=Dur, 2=Title, 3=Art, 4=Genre, 5=Alb, 6=ID"""
    if not songs or not cpp_lib: return songs

    album_map = get_album_map(albums)
    c_arr = songs_to_c(songs, album_map)

```

```

# Виклик C++ (QuickSort)
cpp_lib.sort_songs(c_arr, len(songs), mode, 1 if ascending else 0)

# Відновлення порядку
song_map = {s.id: s for s in songs}
res = []
for i in range(len(songs)):
    if c_arr[i].id in song_map:
        res.append(song_map[c_arr[i].id])
return res

def mergesort_albums_by_year(albums: List[Album]) -> List[Album]:
    if not albums or not cpp_lib: return albums
    c_arr = albums_to_c(albums)

    # Виклик C++ (MergeSort)
    cpp_lib.sort_albums_by_year(c_arr, len(albums))

    album_map = {a.id: a for a in albums}
    res = []
    for i in range(len(albums)):
        if c_arr[i].id in album_map:
            res.append(album_map[c_arr[i].id])
    return res

def binary_search_album(albums: List[Album], query_title: str) -> Optional[Album]:
    """
    Виконує бінарний пошук альбому за назвою через C++.
    УВАГА: Список albums має бути попередньо відсортований за назвою!
    """
    if not albums or not cpp_lib: return None

    # Конвертуємо в C-структури
    c_arr = albums_to_c(albums)
    query_bytes = query_title.encode('utf-8')

    # Виклик C++
    found_id = cpp_lib.binary_search_album(c_arr, len(albums), query_bytes)

    if found_id != -1:
        for a in albums:
            if a.id == found_id:
                return a
    return None

def greedy_top_n_songs(songs: List[Song], n: int) -> List[Song]:
    if not songs or not cpp_lib: return songs[:n]
    c_arr = songs_to_c(songs, {})

    # Виклик C++ (Greedy)
    cpp_lib.get_top_n_greedy(c_arr, len(songs), n)

```

```

song_map = {s.id: s for s in songs}
res = []
for i in range(min(n, len(songs))):
    if c_arr[i].id in song_map:
        res.append(song_map[c_arr[i].id])
return res

```

gui_main.py: Керує логікою головного вікна, навігацією між розділами та відображенням даних у таблицях.

```

import sys
import time
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QLineEdit, QTableWidgetItem,
                             QMessageBox,
                             QHeaderView, QStackedWidget, QLabel, QListWidget, QFrame,
                             QComboBox,
                             QInputDialog, QListWidgetItem, QTextEdit, QDialog,
                             QFormLayout)
from PyQt5.QtCore import QSettings, Qt

from storage import (init_db, add_song, get_all_songs, delete_song_by_id, update_song,
                     add_album, get_all_albums, delete_album_by_id, update_album,
                     get_album_songs,
                     create_playlist, get_playlists, add_song_to_playlist,
                     get_playlist_songs,
                     get_or_create_default_fan, update_fan_profile,
                     get_all_artists, update_artist_details, add_artist_to_db,
                     delete_artist_by_name)

from dialogs import SongDialog, AlbumDialog
from algorithms import (sort_songs_wrapper, search_songs_advanced,
                        greedy_top_n_songs, mergesort_albums_by_year,
                        binary_search_album)
from models import Song, MusicApp, Album, RockSong, PopSong, ClassicalSong, Artist

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Music Fan Directory")
        self.resize(1250, 800)
        self.settings = QSettings("MyCourseWork", "MusicApp")

        init_db()
        self.app_logic = MusicApp()
        self.sort_asc = {}
        self.current_playlist_id = None

        # Завантаження профілю користувача
        try:
            self.app_logic.current_user = get_or_create_default_fan()
        except Exception as e:

```

```

        print(f"Error loading user: {e}")

# Головний віджет
main_widget = QWidget()
self.setCentralWidget(main_widget)
main_layout = QHBoxLayout(main_widget)
main_layout.setContentsMargins(0, 0, 0, 0)
main_layout.setSpacing(0)

# --- 1. SIDEBAR (ЛІВА ПАНЕЛЬ) ---
self.sidebar = QFrame()
self.sidebar.setFixedWidth(240)
self.sidebar_layout = QVBoxLayout(self.sidebar)
self.sidebar_layout.setContentsMargins(15, 25, 15, 25)
self.sidebar_layout.setSpacing(10)

# Логотип
self.lbl_logo = QLabel("🎵 FAN CABINET")
self.lbl_logo.setObjectName("lbl_logo")
self.lbl_logo.setAlignment(Qt.AlignLeft | Qt.AlignVCenter)
self.lbl_logo.setFixedHeight(40)
self.sidebar_layout.addWidget(self.lbl_logo)

# Навігація
self.nav_list = QListWidget()
self.nav_list.setObjectName("nav_list")
self.nav_list.setFixedHeight(320)
self.nav_list.setVerticalScrollBarPolicy(Qt.ScrollBarAlwaysOff)

self.nav_list.addItem(" Songs Library")
self.nav_list.addItem(" Albums")
self.nav_list.addItem(" Artists")
self.nav_list.addItem(" Recommendations 🌟")
self.nav_list.addItem(" Coursework Tech")
self.nav_list.addItem(" 👤 User Profile")

self.sidebar_layout.addWidget(self.nav_list)

# Плейлисти
lbl_pl = QLabel("YOUR PLAYLISTS")
lbl_pl.setObjectName("lbl_pl")
self.sidebar_layout.addWidget(lbl_pl)

self.playlist_list = QListWidget()
self.playlist_list.setObjectName("playlist_list")
self.playlist_list.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)
self.sidebar_layout.addWidget(self.playlist_list)

# Обробники кліків меню
self.nav_list.currentRowChanged.connect(self.switch_page)
self.playlist_list.itemClicked.connect(self.on_playlist_clicked)

# Кнопки внизу сайдбару
self.sidebar_layout.addStretch()

```

```

btn_new_pl = QPushButton("+ New Playlist")
btn_new_pl.clicked.connect(self.create_playlist_action)
self.sidebar_layout.addWidget(btn_new_pl)

self.sidebar_layout.addSpacing(10)

self.btn_theme = QPushButton("🌈 Switch Theme")
self.btn_theme.clicked.connect(self.toggle_theme)
self.sidebar_layout.addWidget(self.btn_theme)

main_layout.addWidget(self.sidebar)

# --- 2. MAIN CONTENT (ПРАВА ЧАСТИНА) ---
self.pages = QStackedWidget()
main_layout.addWidget(self.pages)

# Ініціалізація сторінок
self.page_songs = QWidget()
self.page_albums = QWidget()
self.page_artists = QWidget()
self.page_recs = QWidget()
self.page_tech = QWidget()
self.page_profile = QWidget()
self.page_single_playlist = QWidget()

self.pages.addWidget(self.page_songs) # Index 0
self.pages.addWidget(self.page_albums) # Index 1
self.pages.addWidget(self.page_artists) # Index 2
self.pages.addWidget(self.page_recs) # Index 3
self.pages.addWidget(self.page_tech) # Index 4
self.pages.addWidget(self.page_profile) # Index 5
self.pages.addWidget(self.page_single_playlist) # Index 6

# Налаштування UI для кожної сторінки
self.setup_songs_ui()
self.setup_albums_ui()
self.setup_artists_ui()
self.setup_recs_ui()
self.setup_tech_ui()
self.setup_profile_ui()
self.setup_playlist_ui()

# Завантаження даних
self.load_songs()
self.load_albums()
self.load_artists()
self.load_playlists()
self.load_theme_state()

def switch_page(self, index):
    # Якщо клікнули на основне меню, знімаємо виділення з плейлистів
    if self.nav_list.hasFocus():
        self.playlist_list.setCurrentItem(None)
        self.current_playlist_id = None

    # Оновлюємо дані при перемиканні

```

```

        if index == 1: self.load_albums()
        if index == 2: self.load_artists()
        if index == 3: self.refresh_recs()

    self.pages.setCurrentIndex(index)

# ===== SONGS PAGE =====
def setup_songs_ui(self):
    layout = QVBoxLayout(self.page_songs)
    layout.setContentsMargins(0, 0, 0, 0)
    layout.setSpacing(0)

    # Header
    header = QFrame()
    header.setFixedHeight(80)
    header.setObjectName("header_frame")
    h1 = QHBoxLayout(header)
    h1.setContentsMargins(30, 20, 30, 20)

    title = QLabel("Songs Library")
    title.setObjectName("page_title")
    h1.addWidget(title)

    h1.addStretch()

    # Пошук (Лінійний)
    self.search_type = QComboBox()
    self.search_type.addItems(["All", "Title", "Artist", "Genre"])
    self.search_type.setFixedWidth(120)
    self.search_type.setCursor(Qt.PointingHandCursor)
    h1.addWidget(self.search_type)

    lbl_search = QLabel("🔍")
    lbl_search.setStyleSheet("font-size: 20px; color: #888;")
    h1.addWidget(lbl_search)

    self.search_input = QLineEdit()
    self.search_input.setPlaceholderText("Search songs (Linear)...")
    self.search_input.textChanged.connect(self.search_song_live)
    self.search_input.setFixedWidth(250)
    self.search_input.setFixedHeight(35)
    h1.addWidget(self.search_input)

    layout.addWidget(header)

    # Таблиця
    content_box = QVBoxLayout()
    content_box.setContentsMargins(30, 20, 30, 20)
    content_box.setSpacing(15)

    self.table_songs = QTableWidgetItem(0, 8)
    cols = ["ID", "Title", "Artist", "Album", "Genre", "Rating", "Style Info (Poly)",
"Rec. Vol"]
    self.table_songs.setHorizontalHeaderLabels(cols)
    self.table_songs.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.table_songs.setSelectionBehavior(QTableWidgetItem.SelectRows)

```

```

self.table_songs.setSelectionMode(QTableWidget.SingleSelection)
self.table_songs.doubleClicked.connect(self.edit_song_action)
self.table_songs.horizontalHeader().sectionClicked.connect(self.on_header_clicked)
content_box.addWidget(self.table_songs)

# Кнопки
controls = QHBoxLayout()

btn_add = QPushButton("+ Add Song")
btn_add.setObjectName("btn_add_song")
btn_add.setFixedSize(140, 45)
btn_add.clicked.connect(self.add_song_action)

btn_edit = QPushButton("Edit")
btn_edit.setFixedSize(100, 45)
btn_edit.clicked.connect(self.edit_song_action)

btn_pl = QPushButton("Add to Playlist")
btn_pl.setFixedSize(140, 45)
btn_pl.clicked.connect(self.add_to_playlist_action)

btn_del = QPushButton("Delete")
btn_del.setObjectName("btn_delete")
btn_del.setFixedSize(100, 45)
btn_del.clicked.connect(self.delete_song_action)

controls.addWidget(btn_add)
controls.addWidget(btn_edit)
controls.addWidget(btn_pl)
controls.addStretch()
controls.addWidget(btn_del)

content_box.addLayout(controls)
layout.addLayout(content_box)

def load_songs(self, songs=None):
    if songs is None:
        self.app_logic.songs = get_all_songs()
        songs = self.app_logic.songs

    # Кешуємо альбоми для швидкого доступу
    albums_dict = {a.id: a.title for a in get_all_albums()}

    self.table_songs.setRowCount(len(songs))
    for i, s in enumerate(songs):
        self.table_songs.setItem(i, 0, QTableWidgetItem(str(s.id)))
        self.table_songs.setItem(i, 1, QTableWidgetItem(s.title))
        self.table_songs.setItem(i, 2, QTableWidgetItem(s.artist))
        # Відображаємо назву альбому замість ID
        alb_name = albums_dict.get(s.album_id, "-")
        self.table_songs.setItem(i, 3, QTableWidgetItem(alb_name))
        self.table_songs.setItem(i, 4, QTableWidgetItem(s.genre))

        pop_text = f"{s.popularity_score}"
        if s.popularity_score >= 9.0: pop_text += " 🔥 "
        self.table_songs.setItem(i, 5, QTableWidgetItem(pop_text))

```

```

        # Поліморфізм: Style Description
        self.table_songs.setItem(i, 6, QTableWidgetItem(s.getStyleDescription()))
        # Поліморфізм: Recommended Volume
        self.table_songs.setItem(i, 7,
QTableWidgetItem(str(s.getRecommendedVolume()))))

def search_song_live(self):
    query = self.search_input.text()
    mode = self.search_type.currentIndex()
    if not query:
        self.load_songs()
        return
    # Використовуємо C++ для лінійного пошуку
    albums = get_all_albums()
    found = search_songs_advanced(self.app_logic.songs, albums, query, mode)
    self.load_songs(found)

def on_header_clicked(self, index):
    # Лоріка сортування пісень через C++ QuickSort
    mode_map = {0: 6, 1: 2, 2: 3, 3: 5, 4: 4, 5: 0} # Mapping індексів колонок
    mode = mode_map.get(index, -1)
    if mode == -1: return

    current_asc = self.sort_asc.get(index, True)
    self.sort_asc[index] = not current_asc

    albums = get_all_albums()
    sorted_list = sort_songs_wrapper(self.app_logic.songs, albums, mode, current_asc)
    self.load_songs(sorted_list)

def add_song_action(self):
    current_albums = get_all_albums()
    current_artists = get_all_artists() # Отримуємо список артистів

    # Передаємо artists_list у діалог
    dlg = SongDialog(self, albums_list=current_albums, artists_list=current_artists)

    if dlg.exec_():
        try:
            data = dlg.get_song_data()
            final_album_id = data["album_id"]

            # Логіка створення альбому (без змін)
            if final_album_id == 0 and data["album_text"] and "No Album" not in
data["album_text"]]:
                existing = next((a for a in current_albums if a.title.lower() ==
data["album_text"].lower()), None)
                if existing:
                    final_album_id = existing.id
                else:
                    # Важливо використати правильного артиста
                    new_album = Album(title=data["album_text"], artist=data["artist"],
year=2025)
                    final_album_id = add_album(new_album)

```

```

        # Вибір класу пісні (без змін)
        genre = data["genre"].lower()
        if "rock" in genre:
            SongClass = RockSong
        elif "pop" in genre:
            SongClass = PopSong
        elif "classic" in genre:
            SongClass = ClassicalSong
        else:
            SongClass = Song

        args = {k: v for k, v in data.items() if k not in ['album_text',
'album_id']}
        args['album_id'] = final_album_id

        new_song = SongClass(**args)
        add_song(new_song)

        self.load_songs()
        self.load_artists()
        self.refresh_recs()
    except Exception as e:
        QMessageBox.critical(self, "Error", f"Could not add song: {str(e)}")

def edit_song_action(self):
    row = self.table_songs.currentRow()
    if row < 0: return QMessageBox.warning(self, "Warning", "Select a song to edit!")

    try:
        sid = int(self.table_songs.item(row, 0).text())
        target = next((s for s in self.app_logic.songs if s.id == sid), None)

        if target:
            # Тут теж передаємо список артистів
            dlg = SongDialog(self, song=target,
                            albums_list=get_all_albums(),
                            artists_list=get_all_artists())

            if dlg.exec_():
                data = dlg.get_song_data()
                target.title = data['title']
                target.artist = data['artist']
                target.genre = data['genre']
                target.duration_sec = data['duration_sec']
                target.popularity_score = data['popularity_score']
                target.lyrics = data['lyrics']
                target.album_id = data['album_id']

                update_song(target)
                self.load_songs()
                self.load_artists()
    except Exception as e:
        QMessageBox.critical(self, "Error", f"Edit error: {e}")

def delete_song_action(self):
    row = self.table_songs.currentRow()

```

```

if row >= 0:
    sid = int(self.table_songs.item(row, 0).text())
    if QMessageBox.question(self, "Confirm", "Delete this song?",
                            QMessageBox.Yes | QMessageBox.No) == QMessageBox.Yes:
        delete_song_by_id(sid)
        self.load_songs()
        self.load_artists()
        self.refresh_recs()

# ===== ALBUMS PAGE =====
def setup_albums_ui(self):
    layout = QVBoxLayout(self.page_albums)
    layout.setContentsMargins(0, 0, 0, 0)

    # Header
    header = QFrame()
    header.setFixedHeight(80)
    header.setObjectName("header_frame")
    h1 = QHBoxLayout(header)
    h1.setContentsMargins(30, 20, 30, 20)

    title = QLabel("Albums Library")
    title.setObjectName("page_title")
    h1.addWidget(title)
    h1.addStretch()

    # Кнопка Binary Search
    btn_bin = QPushButton("Binary Search (Year)")
    btn_bin.setFixedWidth(200) # Збільшив ширину
    btn_bin.clicked.connect(self.binary_search_album_action)
    h1.addWidget(btn_bin)

    btn_add = QPushButton("+ Add Album")
    btn_add.setObjectName("btn_add_song")
    btn_add.setFixedSize(140, 40)
    btn_add.clicked.connect(self.add_album_action)
    h1.addWidget(btn_add)

    layout.addWidget(header)

    content = QVBoxLayout()
    content.setContentsMargins(30, 20, 30, 20)

    # Таблица: 6 колонок (ID, Title, Artist, Year, User Rating)
    self.table_albums = QTableWidgetItem(0, 5)
    self.table_albums.setHorizontalHeaderLabels(["ID", "Title", "Artist", "Year",
"User Rating"])
    self.table_albums.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.table_albums.setSelectionBehavior(QTableWidgetItem.SelectRows)
    self.table_albums.setSelectionMode(QTableWidgetItem.SingleSelection)
    self.table_albums.doubleClicked.connect(self.show_album_tracks)

    # Підключаємо сортування по року (Mergesort)

self.table_albums.horizontalHeader().sectionClicked.connect(self.on_album_header_clicked)

```

```

content.addWidget(self.table_albums)

# Кнопки
box = QHBoxLayout()
btn_edit = QPushButton("Edit Album & Rating")
btn_edit.setFixedSize(180, 45)
btn_edit.clicked.connect(self.edit_album_action)

btn_tracks = QPushButton("Show Tracks")
btn_tracks.setFixedSize(140, 45)
btn_tracks.clicked.connect(self.show_album_tracks)

btn_del = QPushButton("Delete")
btn_del.setObjectName("btn_delete")
btn_del.setFixedSize(100, 45)
btn_del.clicked.connect(self.delete_album_action)

box.addWidget(btn_tracks)
box.addWidget(btn_edit)
box.addStretch()
box.addWidget(btn_del)

content.addLayout(box)
layout.addLayout(content)

def load_albums(self, albums=None):
    if albums is None: albums = get_all_albums()
    self.app_logic.albums = albums

    self.table_albums.setRowCount(len(albums))
    for i, a in enumerate(albums):
        self.table_albums.setItem(i, 0, QTableWidgetItem(str(a.id)))
        self.table_albums.setItem(i, 1, QTableWidgetItem(a.title))
        self.table_albums.setItem(i, 2, QTableWidgetItem(a.artist))
        self.table_albums.setItem(i, 3, QTableWidgetItem(str(a.year)))

        ur = getattr(a, 'user_rating', 0.0)
        self.table_albums.setItem(i, 4, QTableWidgetItem(f"{ur:.1f}"))

def on_album_header_clicked(self, index):
    if index == 3: # Year column
        # Сортування злиттям (Mergesort) через C++
        sorted_albums = mergesort_albums_by_year(self.app_logic.albums)
        self.load_albums(sorted_albums)
        QMessageBox.information(self, "Sort", "Albums sorted by Year using Mergesort
(C++)")

def binary_search_album_action(self):
    # 1. Сортуємо (обов'язково для бінарного пошуку)
    self.app_logic.albums.sort(key=lambda x: x.title.lower())

    text, ok = QDialog.getDialog(self, "Binary Search", "Enter EXACT Album Title:")
    if ok and text:
        # 2. Шукаємо
        import bisect
        titles = [a.title.lower() for a in self.app_logic.albums]

```

```

        idx = bisect.bisect_left(titles, text.lower())

        if idx != len(titles) and titles[idx] == text.lower():
            found = self.app_logic.albums[idx]
            self.load_albums([found])
            QMessageBox.information(self, "Result", f"Found: {found.title}
({found.year})")
        else:
            QMessageBox.warning(self, "Result", "Not found via Binary Search")
            self.load_albums() # Скидаємо фільтр

def add_album_action(self):
    dlg = AlbumDialog(self)
    if dlg.exec_():
        data = dlg.get_data()
        if data['title']:
            # Також беремо рейтинг одразу з діалогу
            a = Album(
                title=data['title'],
                artist=data['artist'],
                year=data['year'],
                rating=0.0,
                user_rating=data['user_rating']
            )
            add_album(a)
            self.load_albums()
            self.load_artists()

def edit_album_action(self):
    row = self.table_albums.currentRow()
    if row < 0: return QMessageBox.warning(self, "Warning", "Select album!")

    aid = int(self.table_albums.item(row, 0).text())
    target = next((a for a in self.app_logic.albums if a.id == aid), None)

    if target:
        dlg = AlbumDialog(self, album=target)
        if dlg.exec_():
            data = dlg.get_data()
            target.title = data['title']
            target.artist = data['artist']
            target.year = data['year']
            target.user_rating = data['user_rating']

            update_album(target)
            self.load_albums()
            self.load_artists()

def show_album_tracks(self):
    row = self.table_albums.currentRow()
    if row < 0: return
    aid = int(self.table_albums.item(row, 0).text())
    title = self.table_albums.item(row, 1).text()

    # Отримуємо пісні з БД
    songs = get_album_songs(aid)

```

```

if songs:
    msg = "<ul>" + "".join([f"<li>{s}</li>" for s in songs]) + "</ul>"
else:
    msg = "<i>No songs in this album.</i>"

QMessageBox.information(self, f"Album: {title}", f"<h3>Tracks:</h3>{msg}")

def delete_album_action(self):
    row = self.table_albums.currentRow()
    if row >= 0:
        aid = int(self.table_albums.item(row, 0).text())
        if QMessageBox.question(self, "Confirm", "Delete this album?",
                                QMessageBox.Yes | QMessageBox.No) == QMessageBox.Yes:
            delete_album_by_id(aid)
            self.load_albums()
            self.load_songs() # Оновити пісні (бо вони втратили альбом)

# ===== ARTISTS PAGE =====
def setup_artists_ui(self):
    layout = QVBoxLayout(self.page_artists)
    layout.setContentsMargins(0, 0, 0, 0)

    # Header
    header = QFrame()
    header.setFixedHeight(80)
    header.setObjectName("header_frame")
    h1 = QHBoxLayout(header)
    h1.setContentsMargins(30, 20, 30, 20)

    title = QLabel("Artists Library")
    title.setObjectName("page_title")
    h1.addWidget(title)
    layout.addWidget(header)

    content = QVBoxLayout()
    content.setContentsMargins(30, 20, 30, 20)

    # Таблиця Артистів
    self.table_artists = QTableWidgetItem(0, 3)
    self.table_artists.setHorizontalHeaderLabels(["Name", "Genre", "Country"])
    self.table_artists.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.table_artists.setSelectionBehavior(QTableWidgetItem.SelectRows)
    self.table_artists.setSelectionMode(QTableWidgetItem.SingleSelection)
    self.table_artists.doubleClicked.connect(self.edit_artist_action)
    content.addWidget(self.table_artists)

    btn_box = QHBoxLayout()

    btn_add = QPushButton("+ Add Artist")
    btn_add.setObjectName("btn_add_song")
    btn_add.setFixedSize(140, 45)
    btn_add.clicked.connect(self.add_artist_action)
    btn_box.addWidget(btn_add)

    btn_edit = QPushButton("View Albums / Edit Info")

```

```

btn_edit.setFixedSize(220, 45)
btn_edit.clicked.connect(self.edit_artist_action)

btn_del_art = QPushButton("Delete Artist")
btn_del_art.setObjectName("btn_delete")
btn_del_art.setFixedSize(120, 45)
btn_del_art.clicked.connect(self.delete_artist_action)

btn_box.addStretch()
btn_box.addWidget(btn_edit)
btn_box.addWidget(btn_del_art) # Додаємо кнопку в макет

content.addLayout(btn_box)
layout.addLayout(content)

def add_artist_action(self):
    d = QDialog(self)
    d.setWindowTitle("Add New Artist")
    d.setMinimumWidth(300)

    form = QFormLayout(d)

    name_input = QLineEdit()
    genre_input = QLineEdit()
    country_input = QLineEdit()

    form.addRow("Name:", name_input)
    form.addRow("Genre:", genre_input)
    form.addRow("Country:", country_input)

    btn_layout = QHBoxLayout()
    btn_ok = QPushButton("Add")
    btn_ok.clicked.connect(d.accept)
    btn_ok.setStyleSheet("background-color: #198754; color: white;") # Зелена

    btn_cancel = QPushButton("Cancel")
    btn_cancel.clicked.connect(d.reject)

    btn_layout.addWidget(btn_ok)
    btn_layout.addWidget(btn_cancel)
    form.addRow(btn_layout)

    if d.exec_():
        name = name_input.text().strip()
        genre = genre_input.text().strip()
        country = country_input.text().strip()

        if not name:
            QMessageBox.warning(self, "Error", "Name is required!")
            return

        try:
            add_artist_to_db(name, genre, country)
            self.load_artists()
            QMessageBox.information(self, "Success", f"Artist '{name}' added!")
        except Exception as e:

```

```

        QMessageBox.critical(self, "Error", str(e))

def load_artists(self):
    try:
        artists = get_all_artists()
        self.app_logic.artists = artists
        self.table_artists.setRowCount(len(artists))
        for i, art in enumerate(artists):
            self.table_artists.setItem(i, 0, QTableWidgetItem(art.name))
            self.table_artists.setItem(i, 1, QTableWidgetItem(art.genre))
            self.table_artists.setItem(i, 2, QTableWidgetItem(art.country))
    except Exception as e:
        print(f"Artist load error: {e}")

def edit_artist_action(self):
    row = self.table_artists.currentRow()
    if row < 0: return QMessageBox.warning(self, "Info", "Select an artist first")

    name = self.table_artists.item(row, 0).text()
    target = next((a for a in self.app_logic.artists if a.name == name), None)

    if target:
        # Створюємо діалог для редагування деталей та перегляду альбомів
        d = QDialog(self)
        d.setWindowTitle(f"Details: {name}")
        d.setMinimumWidth(400)
        form = QFormLayout(d)

        # Поля для редагування
        gen_edit = QLineEdit(target.genre)
        ctr_edit = QLineEdit(target.country)

        form.addRow("Genre:", gen_edit)
        form.addRow("Country:", ctr_edit)

        # СПИСОК АЛЬБОМІВ
        lbl_albums = QLabel("Albums by this Artist:")
        lbl_albums.setStyleSheet("font-weight: bold; margin-top: 10px;")
        form.addRow(lbl_albums)

        albums_list = QListWidget()
        # Знаходимо всі альбоми цього артиста
        artist_albums = [a for a in get_all_albums() if a.artist == name]

        if artist_albums:
            for a in artist_albums:
                albums_list.addItem(f"{a.title} ({a.year})")
        else:
            albums_list.addItem("No albums found.")

        form.addRow(albums_list)

        # Кнопки
        bbox = QHBoxLayout()
        b_save = QPushButton("Save Info")
        b_save.setStyleSheet("background-color: #e49ed0; color: white;")

```

```

        b_save.clicked.connect(d.accept)

        b_close = QPushButton("Close")
        b_close.clicked.connect(d.reject)

        bbox.addWidget(b_save)
        bbox.addWidget(b_close)
        form.addRow(bbox)

        # Якщо натиснули Save
        if d.exec_():
            target.genre = gen_edit.text()
            target.country = ctr_edit.text()
            try:
                update_artist_details(target)
                self.load_artists()
                QMessageBox.information(self, "Success", "Artist details updated!")
            except Exception as e:
                QMessageBox.warning(self, "Error", f"Save failed: {e}")

    def delete_artist_action(self):
        row = self.table_artists.currentRow()
        if row < 0: return QMessageBox.warning(self, "Info", "Select an artist to delete")

        name = self.table_artists.item(row, 0).text()

        if QMessageBox.question(self, "Confirm",
                                f"Delete artist '{name}'? \n(Songs will remain but won't
be linked to this artist entry)",
                                QMessageBox.Yes | QMessageBox.No) == QMessageBox.Yes:
            delete_artist_by_name(name)
            self.load_artists()
            QMessageBox.information(self, "Success", "Artist deleted.")

    # RECOMMENDATIONS
    def setup_recs_ui(self):
        layout = QVBoxLayout(self.page_recs)
        lbl = QLabel("Recommendations 🌟")
        lbl.setStyleSheet("font-size: 24px; font-weight: bold; color: #ff70db; margin:
20px;")
        layout.addWidget(lbl, alignment=Qt.AlignCenter)

        self.recs_label = QLabel("Add songs to see recommendations!")
        self.recs_label.setStyleSheet("font-size: 16px; margin: 20px;")
        self.recs_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.recs_label)
        layout.addStretch()

    def refresh_recs(self):
        songs = get_all_songs()
        if not songs: return
        # Жадібний алгоритм: бере топ-5 найкращих
        top = greedy_top_n_songs(songs, 5)

        txt = "<h2 style='text-align:center;'>Top 5 Trending Songs</h2>"
        for i, s in enumerate(top):

```

```

        desc = s.getStyleDescription()
        txt += f"<p style='text-align:center; font-size:14px;'>{i + 1}.
<b>{s.title}</b> - {s.artist} <br><span style='color:#ff70db'>Rating:
{s.popularity_score}</span> | <i>{desc}</i></p>"
        self.recs_label.setText(txt)

# TECH CONSOLE
def setup_tech_ui(self):
    layout = QVBoxLayout(self.page_tech)
    lbl = QLabel("Technical Coursework Requirements")
    lbl.setStyleSheet("font-size: 24px; font-weight: bold; margin: 20px;")
    layout.addWidget(lbl)

    self.tech_console = QTextEdit()
    self.tech_console.setReadOnly(True)
    self.tech_console.setStyleSheet(
        "background-color: #1e1e1e; color: #00ff00; font-family: Consolas; font-size:
16px; padding: 10px;")
    layout.addWidget(self.tech_console)

    controls = QFrame()
    controls.setStyleSheet("background-color: #ccc; border-radius: 5px; margin:
10px;")
    cl = QHBoxLayout(controls)

    b1 = QPushButton("1. OOP & Poly")
    b1.clicked.connect(self.run_tech_oop)
    b1.setStyleSheet("background-color: #e49ed0; color: white;")

    b2 = QPushButton("2. Sorting")
    b2.clicked.connect(self.run_tech_sorting)
    b2.setStyleSheet("background-color: #E91E63; color: white;")

    b3 = QPushButton("3. Linear Search (C++)")
    b3.setStyleSheet("background-color: #ff9800; color: white;")
    b3.clicked.connect(self.run_tech_search)

    b4 = QPushButton("4. Binary Search")
    b4.setStyleSheet("background-color: #177822; color: white;")
    b4.clicked.connect(self.run_tech_binary_search)

    cl.addWidget(b1)
    cl.addWidget(b2)
    cl.addWidget(b3)
    cl.addWidget(b4)
    layout.addWidget(controls)

def log_tech(self, text):
    self.tech_console.append(f">> {text}")

def run_tech_oop(self):
    self.tech_console.clear()
    self.log_tech(">>> Testing OOP Polymorphism (Dynamic Binding)...")

    all_songs = get_all_songs()
    # Шукаємо в базі хоча б по одному представнику різних класів

```

```

showcase = []
types_found = set()
for s in all_songs:
    s_type = type(s).__name__
    if s_type not in types_found:
        showcase.append(s)
        types_found.add(s_type)

if len(showcase) < 2:
    showcase.append(RockSong(title="Demo Rock", artist="System of a Down"))
    showcase.append(ClassicalSong(title="Demo Classic", artist="Mozart"))

for s in showcase:
    self.log_tech(f"Object: '{s.title}' | Class: {type(s).__name__}")
    self.log_tech(f"Style Description: {s.getStyleDescription()}")
    self.log_tech(f"Recommended Volume: {s.getRecommendedVolume()}%")
    self.log_tech("-" * 25)

def run_tech_sorting(self):
    self.tech_console.clear()
    self.log_tech(">>> Testing C++ Quicksort (Sorting by Rating)...")
    songs = get_all_songs()
    if not songs: return self.log_tech("Error: Song database is empty.")

    start = time.perf_counter()
    # Call C++ wrapper (0 = popularity mode, False = descending)
    sorted_list = sort_songs_wrapper(songs, get_all_albums(), 0, False)
    end = time.perf_counter()

    duration = (end - start) * 1000
    self.log_tech(f"Sorted {len(songs)} items in {duration:.4f} ms.")
    self.log_tech("TOP-5 Songs after sorting:")
    for s in sorted_list[:5]:
        self.log_tech(f" -> {s.title} (Rating: {s.popularity_score})")

def run_tech_search(self):
    text, ok = QDialog.getText(self, "Linear Search", "Enter search query:")
    if ok and text:
        self.tech_console.clear()
        self.log_tech(f">>> C++ Linear Search (Core): '{text}'...")
        songs = get_all_songs()

        start = time.perf_counter()
        # Call C++ filtering logic
        found = search_songs_advanced(songs, get_all_albums(), text, 0)
        end = time.perf_counter()

        duration = (end - start) * 1000
        self.log_tech(f"Found {len(found)} results in {duration:.4f} ms.")
        for s in found:
            self.log_tech(f" [+] Found: {s.title} - {s.artist} ({s.genre})")

def run_tech_binary_search(self):
    text, ok = QDialog.getText(self, "Binary Search", "Exact Album Title:")
    if ok and text:
        self.tech_console.clear()

```

```

        self.log_tech(f"Binary searching for album: {text}...")
        # Prepare data
        albums = get_all_albums()
        albums.sort(key=lambda x: x.title) # Must be sorted

        start = time.perf_counter()
        res = binary_search_album(albums, text)
        end = time.perf_counter()

        if res:
            self.log_tech(f"FOUND: {res.title} ({res.year})")
        else:
            self.log_tech("NOT FOUND.")
            self.log_tech(f"Time: {(end - start) * 1000:.4f} ms")

# PROFILE
def setup_profile_ui(self):
    layout = QVBoxLayout(self.page_profile)
    layout.setContentsMargins(40, 40, 40, 40)

    title = QLabel("User Profile")
    title.setObjectName("page_title")
    layout.addWidget(title)

    frame = QFrame()
    frame.setObjectName("profile_frame")
    fl = QFormLayout(frame)

    self.prof_name = QLineEdit()
    self.prof_genres = QLineEdit()
    self.prof_bands = QLineEdit()

    # Load user data
    u = self.app_logic.current_user
    if u:
        self.prof_name.setText(u.get_name())
        self.prof_genres.setText(u.get_genres_str())
        self.prof_bands.setText(u.get_bands_str())

    fl.addRow("Name:", self.prof_name)
    fl.addRow("Favorite Genres:", self.prof_genres)
    fl.addRow("Favorite Bands:", self.prof_bands)

    layout.addWidget(frame)

    btn_save = QPushButton("Save Profile")
    btn_save.setFixedSize(150, 45)
    btn_save.setStyleSheet("background-color: #e49ed0; color: white; font-weight:
bold;")
    btn_save.clicked.connect(self.save_profile_action)

    layout.addWidget(btn_save, alignment=Qt.AlignRight)
    layout.addStretch()

def save_profile_action(self):
    u = self.app_logic.current_user

```

```

    if u:
        u.set_name(self.prof_name.text())
        u.set_genres_from_str(self.prof_genres.text())
        u.set_bands_from_str(self.prof_bands.text())
        try:
            update_fan_profile(u)
            QMessageBox.information(self, "Success", "Profile Updated!")
        except Exception as e:
            QMessageBox.critical(self, "Error", str(e))

# PLAYLISTS
def create_playlist_action(self):
    t, ok = QDialog.get_text(self, "New Playlist", "Name:")
    if ok and t:
        create_playlist(t)
        self.load_playlists()

def load_playlists(self):
    self.playlist_list.clear()
    for p in get_playlists():
        item = QListWidgetItem(p['title'])
        item.setData(Qt.UserRole, p['id'])
        self.playlist_list.addItem(item)

def add_to_playlist_action(self):
    row = self.table_songs.currentRow()
    if row < 0: return QMessageBox.warning(self, "Info", "Select a song first")
    sid = int(self.table_songs.item(row, 0).text())

    pls = get_playlists()
    if not pls: return QMessageBox.warning(self, "Info", "Create a playlist first!")

    names = [p['title'] for p in pls]
    pname, ok = QDialog.get_item(self, "Add to Playlist", "Select Playlist:",
names, 0, False)
    if ok and pname:
        pid = next(p['id'] for p in pls if p['title'] == pname)
        add_song_to_playlist(pid, sid)
        QMessageBox.information(self, "Success", f"Added to {pname}")

def on_playlist_clicked(self, item):
    self.current_playlist_id = item.data(Qt.UserRole)
    self.pages.setCurrentWidget(self.page_single_playlist)
    self.lbl_pl_title.setText(item.text())

    sids = get_playlist_songs(self.current_playlist_id)
    all_s = get_all_songs()
    pl_songs = [s for s in all_s if s.id in sids]
    self.load_playlist_songs(pl_songs)

def setup_playlist_ui(self):
    layout = QVBoxLayout(self.page_single_playlist)
    layout.setContentsMargins(30, 20, 30, 20)

    self.lbl_pl_title = QLabel("Playlist")
    self.lbl_pl_title.setObjectName("page_title")

```

```

layout.addWidget(self.lbl_pl_title)

self.table_pl = QTableWidgetItem(0, 3)
self.table_pl.setHorizontalHeaderLabels(["Title", "Artist", "Duration"])
self.table_pl.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
self.table_pl.setSelectionBehavior(QTableWidgetItem.SelectRows)
layout.addWidget(self.table_pl)

btn_back = QPushButton("Back to Library")
btn_back.setFixedSize(150, 45)
btn_back.clicked.connect(lambda: self.nav_list.setCurrentRow(0))
layout.addWidget(btn_back)

def load_playlist_songs(self, songs):
    self.table_pl.setRowCount(len(songs))
    for i, s in enumerate(songs):
        self.table_pl.setItem(i, 0, QTableWidgetItem(s.title))
        self.table_pl.setItem(i, 1, QTableWidgetItem(s.artist))
        mins = s.duration_sec // 60
        secs = s.duration_sec % 60
        self.table_pl.setItem(i, 2, QTableWidgetItem(f"{mins}:{secs:02d}"))

# THEME
def load_theme_state(self):
    self.is_dark_mode = self.settings.value("is_dark", False, type=bool)
    self.apply_theme()

def toggle_theme(self):
    self.is_dark_mode = not self.is_dark_mode
    self.settings.setValue("is_dark", self.is_dark_mode)
    self.apply_theme()

def apply_theme(self):
    app = QApplication.instance()
    if self.is_dark_mode:
        # DARK THEME
        app.setStyleSheet("""
#444; }

        QMainWindow { background-color: #121212; color: white; }
        QFrame#profile_frame { background-color: #333; border-radius: 10px; }
        QLineEdit { background-color: #282828; color: white; border: 1px solid

        QLabel { color: white; background: transparent; }
        QLabel#lbl_logo { background-color: #181818; color: white; font-weight:
bold; font-size: 22px; }
        QLabel#page_title { font-weight: bold; color: white; font-size: 28px; }

        QListWidget { background-color: #181818; border: none; font-size: 16px;
color: white; outline: none; }
        QListWidget::item { padding: 14px; color: #b3b3b3; border: none; }
        QListWidget::item:selected { background-color: #282828; color: white;
border-left: 4px solid #ff70db; }

        QTableWidgetItem { background-color: #121212; gridline-color: #333; color:
white; border: none; }
        QHeaderView::section { background-color: #181818; color: #b3b3b3;
padding: 6px; font-weight: bold; border: none; }

```

```

        QTableCornerButton::section { background-color: #181818; border: none; }
        QTableWidget::item:selected { background-color: #ff70db; color: black; }

        /* --- SCROLLBAR --- */
        QScrollBar:vertical { border: none; background: #181818; width: 10px;
margin: 0px; }
        QScrollBar::handle:vertical { background: #444; min-height: 20px; border-
radius: 5px; }
        QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical { height:
0px; }
        QScrollBar::add-page:vertical, QScrollBar::sub-page:vertical {
background: none; }

        QLineEdit {
            background-color: #282828; color: white;
            border-radius: 5px; padding: 6px; border: 1px solid #444;
            font-size: 16px;
        }

        /* --- COMBO BOX FIX (Виправлення чорного тексту) --- */
        QComboBox {
            background-color: #282828; color: white;
            border: 1px solid #444; padding: 5px; border-radius: 5px; font-size:
16px;
        }

        /* Велике текстове поле (Lyrics), щоб було ТЕМНИМ */
        QTextEdit {
            background-color: #282828; color: white;
            border-radius: 5px; padding: 6px; border: 1px solid #444;
            font-size: 16px;
        }

        /* Поле вводу всередині комбобоксу */
        QComboBox QLineEdit {
            background-color: #282828; color: white; border: none; font-size:
16px;
        }

        /* Випадаючий список (Dropdown) */
        QComboBox QAbstractItemView {
            background-color: #282828;
            color: white;
            border: 1px solid #444;
            selection-background-color: #ff70db;
            selection-color: black;
            outline: none;
            font-size: 16px;
        }

        QPushButton { background-color: #333; color: white; border-radius: 5px;
padding: 8px; font-weight: bold; }
        QPushButton:hover { background-color: #444; }
        QPushButton#btn_add_song { background-color: #ff70db; color: black; }
        QPushButton#btn_delete { background-color: #dc3545; color: white; }

        QDialog { background-color: #2b2b2b; color: white; }

```

```

        QDialog QLabel { color: white; font-size: 14px; }
        QSpinBox, QDoubleSpinBox { background-color: #282828; color: white;
border: 1px solid #444; padding: 5px; }
        """)
    else:
        # LIGHT THEME
        app.setStyleSheet("""
            QMainWindow { background-color: #f5f5f5; color: black; }
            QFrame#profile_frame { background-color: #ffffff; border: 1px solid #ddd;
border-radius: 10px; }
            QLineEdit { background-color: #fff; color: black; border: 1px solid #ccc;
}

            QLabel { color: black; background: transparent; }
            QLabel#lbl_logo { color: black; font-weight: bold; font-size: 22px; }
            QLabel#page_title { font-weight: bold; color: black; font-size: 28px; }

            QListWidget { background-color: #ffffff; border: none; font-size: 16px;
color: black; outline: none; }
            QListWidget::item { padding: 12px; color: #333; }
            QListWidget::item:selected { background-color: #e0e0e0; color: black;
border-left: 4px solid #ff70db; }

            QTableWidget { background-color: white; gridline-color: #eee; color:
black; border: none; }
            QHeaderView::section { background-color: #f9f9f9; color: #555; padding:
6px; font-weight: bold; }
            QTableWidget::item:selected { background-color: #e0e0e0; color: black; }

            QLineEdit { background-color: #fff; color: black; border: 1px solid #ccc;
border-radius: 5px; padding: 6px; }
            QComboBox { background-color: #fff; color: black; border: 1px solid #ccc;
padding: 5px; }

            QPushButton { background-color: #e0e0e0; color: black; border-radius:
5px; padding: 8px; }
            QPushButton:hover { background-color: #d0d0d0; }
            QPushButton#btn_add_song { background-color: #ff70db; color: white; }
            QPushButton#btn_delete { background-color: #ffcdd2; color: #c62828; }

            QDialog { background-color: #ffffff; color: black; }
            QSpinBox, QDoubleSpinBox { background-color: #fff; color: black; border:
1px solid #ccc; padding: 5px; }
        """)

def main():
    app = QApplication(sys.argv)
    w = MainWindow()
    w.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()

```

dialogs.py: Реалізує функціонал діалогових вікон для введення, валідації та редагування характеристик музичних об'єктів.

```
from PyQt5.QtWidgets import (QDialog, QFormLayout, QLineEdit, QSpinBox,
                             QPushButton, QHBoxLayout, QTextEdit, QDoubleSpinBox,
                             QComboBox, QLabel)
from PyQt5.QtCore import Qt

class SongDialog(QDialog):
    def __init__(self, parent=None, song=None, albums_list=None, artists_list=None):
        super().__init__(parent)
        self.setWindowTitle("Add / Edit Song")
        self.resize(400, 550)

        layout = QFormLayout(self)

        # 1. Title
        self.title_edit = QLineEdit()
        layout.addRow("Title:", self.title_edit)

        # 2. Artist (ВИПАДАЮЧИЙ СПИСОК)
        self.artist_combo = QComboBox()
        self.artist_combo.setEditable(True) # Можна і вибрати, і ввести нового
        self.artist_combo.setInsertPolicy(QComboBox.NoInsert)

        # Налаштування автодоповнення для артистів

        self.artist_combo.completer().setCompletionMode(self.artist_combo.completer().PopupCompletion)
        self.artist_combo.completer().setFilterMode(Qt.MatchContains)

        if artists_list:
            # Сортуємо, щоб легше було шукати очима, і додаємо в список
            sorted_artists = sorted([a.name for a in artists_list])
            self.artist_combo.addItem(sorted_artists)

        layout.addRow("Artist:", self.artist_combo)

        # 3. Album
        self.album_combo = QComboBox()
        self.album_combo.setEditable(True)
        self.album_combo.setInsertPolicy(QComboBox.NoInsert)

        self.album_combo.completer().setCompletionMode(self.album_combo.completer().PopupCompletion)
        self.album_combo.completer().setFilterMode(Qt.MatchContains)

        self.album_combo.addItem("Single", 0)

        if albums_list:
            for album in albums_list:
                display_text = f"{album.title}"
                if album.year > 0: display_text += f" ({album.year})"
```

```

        self.album_combo.addItem(display_text, album.id)

    layout.addRow("Album:", self.album_combo)

    # 4. Duration
    dur_layout = QHBoxLayout()
    self.min_spin = QSpinBox()
    self.min_spin.setRange(0, 59)
    self.min_spin.setSuffix(" m")
    self.sec_spin = QSpinBox()
    self.sec_spin.setRange(0, 59)
    self.sec_spin.setSuffix(" s")
    dur_layout.addWidget(self.min_spin)
    dur_layout.addWidget(self.sec_spin)
    layout.addRow("Duration:", dur_layout)

    # 5. Genre
    self.genre_edit = QLineEdit()
    layout.addRow("Genre:", self.genre_edit)

    # 6. Popularity
    self.popularity_spin = QDoubleSpinBox()
    self.popularity_spin.setRange(0.0, 10.0)
    self.popularity_spin.setDecimals(1)
    layout.addRow("Popularity (0-10):", self.popularity_spin)

    # 7. Lyrics
    self.lyrics_edit = QTextEdit()
    layout.addRow("Lyrics:", self.lyrics_edit)

    # Buttons
    btn_layout = QHBoxLayout()
    self.btn_ok = QPushButton("OK")
    self.btn_cancel = QPushButton("Cancel")
    self.btn_ok.setStyleSheet("background-color: #198754; color: white; padding:
5px;")
    self.btn_cancel.setStyleSheet("background-color: #dc3545; color: white; padding:
5px;")
    btn_layout.addWidget(self.btn_ok)
    btn_layout.addWidget(self.btn_cancel)
    layout.addRow(btn_layout)

    self.btn_ok.clicked.connect(self.accept)
    self.btn_cancel.clicked.connect(self.reject)

    if song:
        self.title_edit.setText(song.title)
        self.artist_combo.setCurrentText(song.artist)
        self.genre_edit.setText(song.genre)
        self.popularity_spin.setValue(song.popularity_score)
        self.lyrics_edit.setPlainText(song.lyrics)

        mins = song.duration_sec // 60
        secs = song.duration_sec % 60
        self.min_spin.setValue(mins)
        self.sec_spin.setValue(secs)

```

```

        index = self.album_combo.findData(song.album_id)
        if index >= 0:
            self.album_combo.setCurrentIndex(index)
        else:
            self.album_combo.setCurrentIndex(0)

def get_song_data(self):
    total_sec = (self.min_spin.value() * 60) + self.sec_spin.value()
    return {
        "title": self.title_edit.text().strip(),
        "artist": self.artist_combo.currentText().strip(),
        "duration_sec": total_sec,
        "genre": self.genre_edit.text().strip(),
        "popularity_score": self.popularity_spin.value(),
        "lyrics": self.lyrics_edit.toPlainText().strip(),
        "album_id": self.album_combo.currentData(),
        "album_text": self.album_combo.currentText().strip()
    }

class AlbumDialog(QDialog):
    def __init__(self, parent=None, album=None):
        super().__init__(parent)
        self.setWindowTitle("Add / Edit Album")
        self.resize(350, 250)

        layout = QFormLayout(self)

        self.title_edit = QLineEdit()
        layout.addRow("Title:", self.title_edit)

        self.artist_edit = QLineEdit()
        layout.addRow("Artist:", self.artist_edit)

        self.year_spin = QSpinBox()
        self.year_spin.setRange(1500, 2026)
        self.year_spin.setValue(2025)
        layout.addRow("Year:", self.year_spin)

        # ПОЛЕ РЕЙТИНГУ
        self.rating_spin = QDoubleSpinBox()
        self.rating_spin.setRange(0.0, 10.0)
        self.rating_spin.setDecimals(1)
        self.rating_spin.setSingleStep(0.5)
        layout.addRow("User Rating:", self.rating_spin)

        btn_layout = QHBoxLayout()
        self.btn_ok = QPushButton("OK")
        self.btn_cancel = QPushButton("Cancel")
        self.btn_ok.setStyleSheet("background-color: #198754; color: white;")
        self.btn_cancel.setStyleSheet("background-color: #dc3545; color: white;")

        btn_layout.addWidget(self.btn_ok)
        btn_layout.addWidget(self.btn_cancel)
        layout.addRow(btn_layout)

```

```

self.btn_ok.clicked.connect(self.accept)
self.btn_cancel.clicked.connect(self.reject)

if album:
    self.title_edit.setText(album.title)
    self.artist_edit.setText(album.artist)
    self.year_spin.setValue(album.year)
    # Встановлюємо поточний рейтинг, якщо він є
    current_rating = getattr(album, 'user_rating', 0.0)
    self.rating_spin.setValue(current_rating)

def get_data(self):
    return {
        "title": self.title_edit.text().strip(),
        "artist": self.artist_edit.text().strip(),
        "year": self.year_spin.value(),
        "user_rating": self.rating_spin.value()
    }

```

Взаємодія з базою даних SQLite

storage.py: Забезпечує взаємодію з базою даних SQLite для надійного збереження та опрацювання музичної інформації.

```
from dataclasses import dataclass
from typing import List

@dataclass
class Song:
    id: int = 0
    title: str = ""
    artist: str = ""
    duration_sec: int = 0
    genre: str = ""
    lyrics: str = ""
    popularity_score: float = 0.0 # Світовий рейтинг
    album_id: int = 0

    def getStyleDescription(self) -> str:
        return "General music style"

    def getRecommendedVolume(self) -> int:
        return 50

@dataclass
class RockSong(Song):
    def getStyleDescription(self) -> str:
        return "Rock: Energetic, guitar-driven sound."

    def getRecommendedVolume(self) -> int:
        return 80

@dataclass
class PopSong(Song):
    def getStyleDescription(self) -> str:
        return "Pop: Catchy melody, repetitive structure."

    def getRecommendedVolume(self) -> int:
        return 60

@dataclass
class ClassicalSong(Song):
    def getStyleDescription(self) -> str:
        return "Classical: Orchestral, complex dynamics."

    def getRecommendedVolume(self) -> int:
```

```
return 45
```

```
@dataclass
class Album:
    id: int = 0
    title: str = ""
    artist: str = ""
    year: int = 0
    rating: float = 0.0 # Світовий рейтинг
    user_rating: float = 0.0 # Оцінка користувача (Fan)
    artist_id: int = 0
```

```
@dataclass
class Artist:
    id: int = 0
    name: str = ""
    genre: str = "Unknown"
    country: str = "Unknown"
```

```
class Fan:
    def __init__(self, id: int = 0, name: str = "New User", genres: str = "", bands: str =
    ""):
        self.id = id
        self.__name = name
        genres_str = genres if genres else ""
        bands_str = bands if bands else ""
        self._favorite_genres = [g.strip() for g in genres_str.split(',') if g.strip()]
        self.favorite_bands = [b.strip() for b in bands_str.split(',') if b.strip()]

    def get_name(self) -> str:
        return self.__name

    def set_name(self, name: str):
        if name: self.__name = name

    def get_genres_str(self) -> str:
        return ", ".join(self._favorite_genres)

    def set_genres_from_str(self, genres_str: str):
        self._favorite_genres = [g.strip() for g in genres_str.split(',') if g.strip()]

    def get_bands_str(self) -> str:
        return ", ".join(self.favorite_bands)

    def set_bands_from_str(self, bands_str: str):
        self.favorite_bands = [b.strip() for b in bands_str.split(',') if b.strip()]

class MusicApp:
    def __init__(self):
        self.songs: List[Song] = []
        self.albums: List[Album] = []
```

```
self.artists: List[Artist] = [] # Список артистів  
self.current_user: Fan = None
```